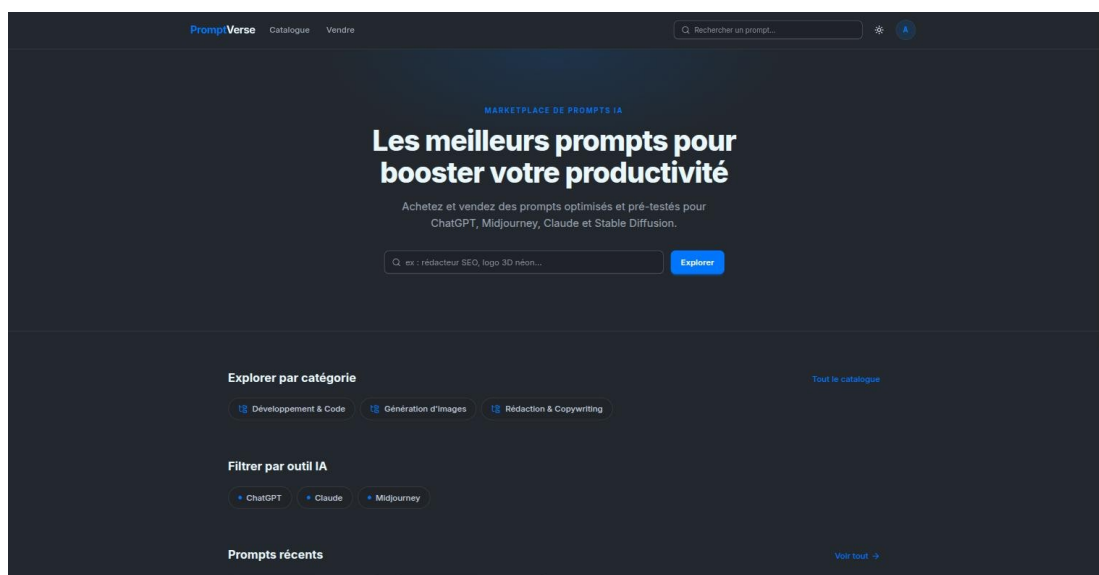


DOSSIER DE PROJET

PromptVerse

Marketplace d'achat et de vente de prompts pour les IA génératives



Candidat : Abduljaber AL OMRAN

Titre professionnel visé : Concepteur Développeur d'Applications (CDA) — niveau 6

Modalité d'accès : Parcours de formation — Simplon

Application en production : <https://promptverse.biz>

Dépôt du code source : <https://github.com/abduljaber23/promptVerse>

Sommaire

1. Compétences du référentiel couvertes par le projet.....	4
2. Résumé du projet.....	5
2.1 Résumé en français.....	5
2.2 Summary in English.....	5
3. Cahier des charges et expression des besoins.....	7
3.1 Contexte et problématique.....	7
3.2 Objectifs du projet.....	7
3.3 Public cible.....	7
3.4 Périmètre fonctionnel.....	8
3.5 Contraintes.....	8
3.6 User stories principales.....	8
4. Gestion de projet.....	10
4.1 Méthodologie.....	10
4.2 Planning.....	10
4.3 Outils de suivi.....	11
4.4 Environnement humain et technique.....	11
4.5 Objectifs de qualité.....	12
5. Spécifications fonctionnelles.....	13
5.1 Les acteurs et leurs droits.....	13
5.2 Diagramme des cas d'utilisation.....	13
5.3 Règles de gestion.....	14
5.4 Maquettes.....	14
5.5 Enchaînement des écrans.....	17
5.6 Diagrammes de séquence.....	18
6. Spécifications techniques.....	20
6.1 Choix technologiques.....	20
6.2 Architecture logicielle.....	20
6.3 Environnement de développement.....	22
6.4 Sécurité de l'application.....	22
6.5 Données personnelles et accessibilité.....	23
7. Conception de la base de données.....	24
7.1 Dictionnaire de données (extrait).....	24
7.2 Modèle conceptuel de données (MCD).....	25
7.3 Modèle logique de données (MLD).....	25
7.4 Modèle physique de données (MPD).....	26
7.5 Diagramme de classes.....	27
7.6 Composants d'accès aux données SQL et NoSQL.....	28
8. Réalisations.....	29
8.1 Back-end.....	29
8.2 Front-end.....	32

8.3 Captures d'écran de l'application.....	34
9. Plan de tests et jeu d'essai.....	38
9.1 Stratégie de tests.....	38
9.2 Tests unitaires.....	38
9.3 Jeu d'essai de la fonctionnalité la plus représentative : l'achat d'un prompt.....	39
9.4 Analyse des écarts.....	40
10. Veille sur les vulnérabilités de sécurité.....	42
10.1 Démarche et sources.....	42
10.2 OWASP Top 10 et mesures appliquées dans PromptVerse.....	42
10.3 Audit des dépendances.....	43
10.4 Vulnérabilité identifiée dans mon propre code.....	43
11. Situation de travail ayant nécessité une recherche anglophone.....	45
11.1 Le problème rencontré.....	45
11.2 La démarche de recherche.....	45
11.3 Extrait de la source et traduction.....	45
11.4 La solution appliquée.....	46
12. Déploiement et démarche DevOps.....	47
12.1 Conteneurisation.....	47
12.2 Intégration et déploiement continu.....	48
12.3 Architecture de production.....	48
12.4 Procédure de déploiement.....	49
13. Bilan et perspectives.....	50
13.1 Bilan du projet.....	50
13.2 Améliorations prévues.....	50
14. Conclusion.....	51
Annexes.....	52
Annexe A — Principales routes de l'API.....	52
Annexe B — Organisation du dépôt.....	53
Annexe C — Glossaire.....	53

1. Compétences du référentiel couvertes par le projet

Le projet PromptVerse couvre les trois activités types du titre professionnel Concepteur Développeur d'Applications. J'ai conçu et développé seul l'ensemble de l'application : chaque compétence ci-dessous a donc été mise en œuvre directement par moi. La dernière colonne indique où trouver les éléments qui la démontrent dans ce dossier.

Activité type	Compétence professionnelle	Mise en œuvre dans PromptVerse	Sections
AT1 — Développer une application sécurisée	Installer et configurer son environnement de travail en fonction du projet	Monorepo Git, Docker Compose (API, client, MySQL, Redis, Mailpit), variables d'environnement validées, ESLint / Prettier	4.4, 6.3
	Développer des interfaces utilisateur	Application React 19 / TypeScript, routes protégées, formulaires validés, maquettes	5.4, 8.2, 8.3
	Développer des composants métier	Publication, masquage du contenu payant, achat Stripe, webhook idempotent, rôles	8.1
	Contribuer à la gestion d'un projet informatique	Sprints Scrum, backlog Trello, spécifications OpenSpec, planning, qualité	4
AT2 — Concevoir et développer une application sécurisée organisée en couches	Analyser les besoins et maquetter une application	Cahier des charges, user stories, cas d'utilisation, wireframes et maquettes	3, 5
	Définir l'architecture logicielle d'une application	Architecture client / API REST en couches, choix technologiques justifiés	6.1, 6.2
	Concevoir et mettre en place une base de données relationnelle	Merise (MCD, MLD, MPD), MySQL 8.4, migrations TypeORM	7.1 à 7.5
	Développer des composants d'accès aux données SQL et NoSQL	Repositories TypeORM (SQL), Redis clé-valeur (NoSQL)	7.6
AT3 — Préparer le déploiement d'une application sécurisée	Préparer et exécuter les plans de tests d'une application	Plan de tests, 18 tests unitaires Jest, jeu d'essai exécuté sur l'API	9
	Préparer et documenter le déploiement d'une application	Dockerfiles multi-stage, compose de production, procédure documentée	12
	Contribuer à la mise en production dans une démarche DevOps	Pipeline GitHub Actions, Docker Hub, VPS, Nginx, HTTPS	12

Deux parties transverses complètent ces compétences : la **veille sur les vulnérabilités de sécurité** (section 10) et une **situation de travail ayant nécessité une recherche sur un site anglophone** (section 11).

2. Résumé du projet

2.1 Résumé en français

PromptVerse est une marketplace web qui permet d'acheter et de vendre des **prompts optimisés pour les IA génératives** (ChatGPT, Midjourney, Claude, DALL·E, Stable Diffusion...). Tout utilisateur inscrit peut à la fois vendre ses propres prompts et acheter ceux des autres. Un prompt payant est réglé avec **Stripe Checkout** ; un prompt dont le prix est fixé à 0 € est directement consultable, sans paiement.

L'application repose sur une architecture découplée : une **API REST NestJS** (TypeScript, TypeORM, MySQL) et une **application monopage React**. La sécurité est prise en compte dès la conception : authentification par JWT dans un cookie httpOnly, contrôle d'accès par rôles, validation systématique des données, limitation de débit et masquage du contenu payant. Le projet est conteneurisé avec Docker, testé automatiquement et déployé par un pipeline GitHub Actions sur un serveur de production accessible à l'adresse <https://promptverse.biz>.

2.2 Summary in English

PromptVerse is a web marketplace where users buy and sell prompts optimized for generative AI tools such as ChatGPT, Midjourney, Claude, DALL·E or Stable Diffusion. There is no separate seller role: every registered user can publish prompts and purchase prompts created by others. Paid prompts are purchased through Stripe Checkout, and a prompt priced at zero is free: its full content is returned directly by the API, without any payment.

The project follows a decoupled, layered architecture. The backend is a REST API built with NestJS and TypeScript, using TypeORM on top of a MySQL 8.4 database. Authentication relies on a JSON Web Token stored in an httpOnly cookie, passwords are hashed with bcrypt, and every route is protected by default by a global guard, with a second guard restricting administration routes by role. Incoming data is validated by DTOs, sensitive routes are rate-limited, and the paid content of a prompt is never sent to users who have not bought it. Stripe payment confirmations are received through a signed, idempotent webhook.

The frontend is a React 19 single-page application written in TypeScript and built with Vite. It uses React Router for navigation and protected routes, TanStack Query for server data, Zustand for client state, and react-hook-form with zod for form validation. The interface is styled with Tailwind CSS.

I designed and developed the whole project on my own, from the requirements and the UML and Merise models to production. Each feature was specified before being coded. The application is containerized with Docker, and a GitHub Actions pipeline runs the tests, builds

the images, pushes them to Docker Hub and deploys them to a VPS behind Nginx with HTTPS.

3. Cahier des charges et expression des besoins

3.1 Contexte et problématique

Les IA génératives sont devenues des outils de travail quotidiens, mais la qualité de leurs résultats dépend fortement de la manière dont on les interroge. Écrire un bon prompt demande du temps, des essais et une connaissance fine de chaque outil. À l'inverse, certains utilisateurs ont développé une vraie expertise et produisent des prompts très efficaces, sans moyen simple de la valoriser.

PromptVerse répond à ces deux besoins en créant une place de marché : les créateurs publient et vendent leurs prompts, les utilisateurs achètent un prompt prêt à l'emploi pour gagner du temps. Le texte complet d'un prompt payant est la valeur vendue : il doit donc rester invisible tant que l'achat n'est pas confirmé.

3.2 Objectifs du projet

- Permettre à un visiteur de parcourir le catalogue et de consulter l'aperçu public de chaque prompt.
- Permettre de créer un compte sécurisé, avec vérification de l'adresse email et réinitialisation du mot de passe.
- Permettre à tout utilisateur connecté de publier un prompt, mis en ligne immédiatement.
- Permettre d'acheter un prompt payant par carte bancaire via Stripe Checkout, et d'accéder librement aux prompts gratuits.
- Garantir que le contenu payant n'est jamais exposé à une personne qui ne l'a pas acheté.
- Offrir aux administrateurs un espace de gestion des utilisateurs, des catégories et des outils IA.
- Livrer une application réellement déployée, testée et reproductible.

3.3 Public cible

- **Utilisateurs d'IA générative** (particuliers, créateurs de contenu, professionnels) qui veulent des prompts prêts à l'emploi.
- **Créateurs de prompts** (« prompt engineers ») qui souhaitent monétiser leur savoir-faire.
- **Débutants** qui préfèrent partir d'exemples fiables plutôt que d'apprendre seuls à écrire des prompts efficaces.

3.4 Périmètre fonctionnel

Inclus dans la version livrée	Hors périmètre (évolutions prévues)
Inscription, vérification d'email, connexion, déconnexion, mot de passe oublié	Connexion via un fournisseur externe (Google, GitHub)
Catalogue paginé, filtre par catégorie et par outil IA, fiche détaillée	Recherche plein texte par mot-clé
Publication d'un prompt avec image de couverture et images d'aperçu	Modération des prompts par l'administration
Achat unitaire par Stripe Checkout, prompts gratuits, historique des achats	Reversement des gains aux vendeurs (Stripe Connect)
Tableau de bord : prompts publiés, ventes, solde, achats, paramètres du compte	Avis, notes et liste de favoris
Administration : utilisateurs et rôles, catégories, outils IA	Notifications en temps réel

3.5 Contraintes

Type	Contrainte
Technique	Stack imposée par le brief chef-d'œuvre : React TypeScript, NestJS, TypeORM, MySQL.
Organisation	Projet réalisé seul, en méthode Agile Scrum, avec une démarche guidée par les spécifications.
Sécurité	Protection des comptes et des paiements, respect des recommandations OWASP, aucun secret dans le code.
Paieement	Aucune donnée bancaire ne transite par l'application : le paiement est entièrement délégué à Stripe.
Données personnelles	Collecte limitée au nécessaire (email, nom d'utilisateur), mot de passe jamais stocké en clair (RGPD).
Déploiement	Application conteneurisée, déployée automatiquement sur un serveur de production en HTTPS.
Délai	Projet mené pendant la période chef-d'œuvre de la formation, en huit sprints.

3.6 User stories principales

Les besoins ont été formalisés en user stories avec des critères d'acceptation. Ces critères ont ensuite servi de base aux tests (section 9).

ID	User story	Critères d'acceptation
US-01	En tant que visiteur, je veux créer un compte afin d'acheter et de vendre des prompts.	Email et nom d'utilisateur uniques ; mot de passe haché ; email de vérification envoyé ; rôle USER attribué.
US-02	En tant qu'utilisateur, je veux me connecter de façon sécurisée.	Connexion refusée si l'email n'est pas vérifié ; jeton placé dans un cookie httpOnly ; trop de tentatives bloquées.

ID	User story	Critères d'acceptation
US-03	En tant qu'utilisateur, je veux publier un prompt afin de le vendre.	Titre, contenu, prix entre 0 et 99,99 €, catégorie et outil IA obligatoires ; images facultatives ; mise en ligne immédiate.
US-04	En tant que visiteur, je veux parcourir le catalogue et filtrer les prompts.	Liste paginée des prompts publiés ; filtre par catégorie et par outil IA ; contenu payant jamais présent dans la liste.
US-05	En tant qu'utilisateur, je veux acheter un prompt payant.	Refus pour son propre prompt, un prompt gratuit ou déjà acheté ; paiement Stripe ; contenu débloqué après confirmation.
US-06	En tant qu'utilisateur, je veux accéder librement à un prompt gratuit.	Contenu visible sans achat ni paiement, y compris pour un visiteur.
US-07	En tant qu'administrateur, je veux gérer les catégories et les outils IA.	Création, modification, archivage et restauration ; accès refusé aux rôles non administrateurs.
US-08	En tant que super-administrateur, je veux modifier le rôle d'un utilisateur.	Action réservée au rôle SUPER_ADMIN ; rôle invalide refusé.

4. Gestion de projet

4.1 Méthodologie

J'ai organisé le projet selon une approche **Agile Scrum adaptée à un développeur seul** : le travail est découpé en huit sprints courts, chacun avec un objectif, un ensemble de tickets et une définition de « terminé ». À la fin de chaque sprint, je vérifiais que les critères d'acceptation étaient respectés avant de passer au suivant.

J'ai combiné cette organisation avec une démarche **guidée par les spécifications** (*spec-driven development*) outillée par **OpenSpec** : avant de coder un module, je rédigeais une proposition versionnée dans le dépôt (besoin, conception, critères d'acceptation, liste de tâches). Le code découle ainsi directement d'une spécification validée, et les critères d'acceptation deviennent la liste des tests à écrire.

L'intelligence artificielle a été utilisée comme **assistant** pour reformuler des spécifications, proposer des pistes de structure et relire du code. Chaque proposition a été vérifiée, testée et adaptée avant d'être intégrée : les choix finaux et la validation sont restés sous ma responsabilité.

4.2 Planning

Le planning ci-dessous a été reconstitué à partir de l'historique Git du dépôt et des documents de sprint. Il montre l'enchaînement réel des phases, de la conception jusqu'à la mise en production.

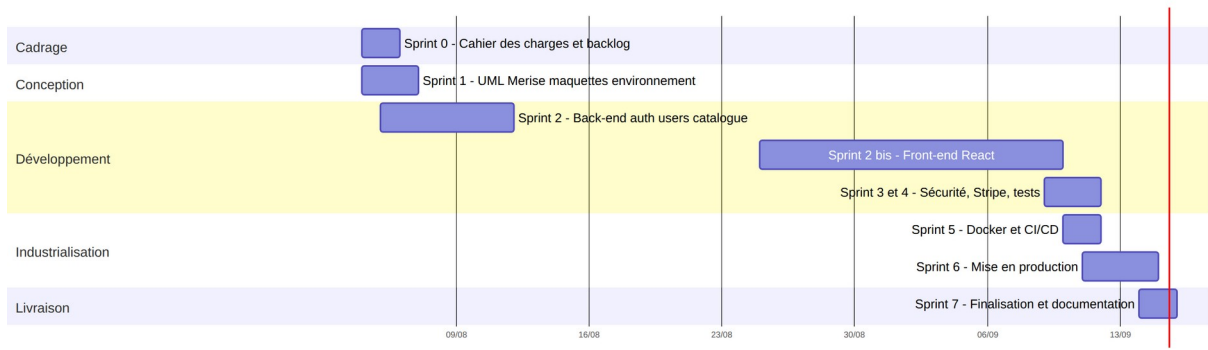


Figure 1 — Planning du projet (diagramme de Gantt)

Sprint	Objectif	Livrables principaux
0	Cadrage	Cahier des charges, rôles, stack technique, backlog, initialisation d'OpenSpec
1	Conception et environnement	Cas d'utilisation, diagrammes de classes et de séquence, Merise, maquettes, Docker Compose
2	Cœur back-end et front-end	Authentification, utilisateurs, catégories, outils IA, prompts, interface React
3	Sécurité, qualité et tests	Guards, rôles, throttling, validation, tests unitaires Jest

Sprint	Objectif	Livrables principaux
4	Fonctionnalités avancées	Achat Stripe Checkout, webhook idempotent, prompts gratuits, administration
5	Industrialisation	Dockerfiles multi-stage, pipeline CI/CD GitHub Actions, images Docker Hub
6	Mise en production	VPS, Nginx, HTTPS Let's Encrypt, volumes persistants, migrations automatiques
7	Finalisation	Simplification du stockage des fichiers, documentation, dossier de projet

4.3 Outils de suivi

- **Trello** : backlog organisé par sprint, avec des cartes contenant la description, une user story au format Gherkin (GIVEN / WHEN / THEN), une checklist et une définition de « terminé ». Le backlog complet est conservé dans [docs/trello_backlog.md](#).
- **OpenSpec** : une proposition par module ([openspec/changes/](#) : auth, users, categories et outils IA, prompts, paiement, Docker et CI/CD...), versionnée avec le code.
- **Git et GitHub** : trois branches — [dev](#) pour le développement, [main](#) pour la version stable et [deploy](#) qui déclenche le déploiement en production. Les messages de commit suivent la convention *Conventional Commits* ([feat](#), [fix](#), [test](#), [refactor](#)...).
- **Documents de sprint** : un fichier par sprint dans [docs/sprints/](#) (objectifs, tâches, compétences mobilisées).

Carte Trello (extrait)	Sprint	Contenu
CARD-0.1 Rédaction du cahier des charges	0	Périmètre, stack, matrice des rôles ; DoD : document validé
CARD-0.2 Modélisation UML et Merise	0	Cas d'utilisation, classes, séquences, MCD / MLD / MPD, dictionnaire de données
CARD-1.1 Spécifications UX/UI et wireframes	1	Charte graphique dark mode, plan du site, wireframes des écrans principaux
CARD-2.2 Authentification JWT et bcrypt	2	Inscription, connexion, guards d'autorisation, sans Passport ni OAuth

4.4 Environnement humain et technique

Environnement humain. J'ai réalisé le projet seul. Les briefs de sprint étaient fournis par mon formateur, avec qui je faisais le point sur l'avancement et les livrables. La documentation officielle des outils (NestJS, Stripe, TypeORM, React) a été ma principale ressource.

Catégorie	Outils
Éditeur et langage	VS Code, TypeScript, Node.js 22
Back-end	NestJS 11, TypeORM, MySQL 8.4, Redis 7, bcrypt, JWT, class-validator, Joi, Multer, Nodemailer

Catégorie	Outils
Front-end	React 19, Vite, React Router 7, TanStack Query, Zustand, react-hook-form, zod, Tailwind CSS, Axios
Conteneurs	Docker, Docker Compose (API, client, MySQL, Redis, Mailpit, RedisInsight)
Tests et API	Jest, Swagger / OpenAPI, Mailpit (emails de développement)
Qualité	ESLint, Prettier, TypeScript (mode strict côté API)
Conception	Mermaid (diagrammes UML), méthode Merise, outil de maquettage, Stitch
Gestion de projet	Trello, OpenSpec, Git, GitHub
DevOps	GitHub Actions, Docker Hub, VPS Linux, Nginx, Let's Encrypt / Certbot

4.5 Objectifs de qualité

- **Code typé et homogène** : TypeScript des deux côtés, ESLint et Prettier exécutés à chaque intégration.
- **Tests automatisés** : les règles métier sensibles (visibilité du contenu, achat, rôles) sont couvertes par des tests unitaires.
- **Porte qualité dans la CI** : un lint ou un test en échec bloque la construction des images et le déploiement.
- **Traçabilité** : chaque fonctionnalité est reliée à une spécification OpenSpec et à des critères d'acceptation.
- **Messages d'erreur exploitables** : l'API renvoie des codes d'erreur explicites (`PROMPT_IS_FREE`, `INVALID_TOKEN`...) que le front-end traduit en messages compréhensibles.
- **Reproductibilité** : le même environnement Docker est utilisé en développement, en intégration continue et en production.

5. Spécifications fonctionnelles

5.1 Les acteurs et leurs droits

L'application distingue un visiteur non connecté et trois rôles, stockés dans une énumération sur l'entité utilisateur. Il n'existe pas de rôle « vendeur » distinct : tout utilisateur peut vendre et acheter, ce qui simplifie le modèle.

Acteur	Description	Droits
Visiteur	Personne non connectée	Parcourir le catalogue, consulter l'aperçu public d'un prompt et le contenu des prompts gratuits, s'inscrire, se connecter.
USER	Acheteur et vendeur	Publier un prompt, acheter un prompt payant, consulter ses achats et ses ventes, gérer son profil et son compte.
ADMIN	Administrateur	Droits USER + consulter les utilisateurs, gérer les catégories et les outils IA (création, modification, archivage, restauration).
SUPER_ADMIN	Super-administrateur	Droits ADMIN + modifier le rôle d'un utilisateur.
Stripe	Système externe	Héberger la page de paiement et confirmer les paiements par webhook.

5.2 Diagramme des cas d'utilisation

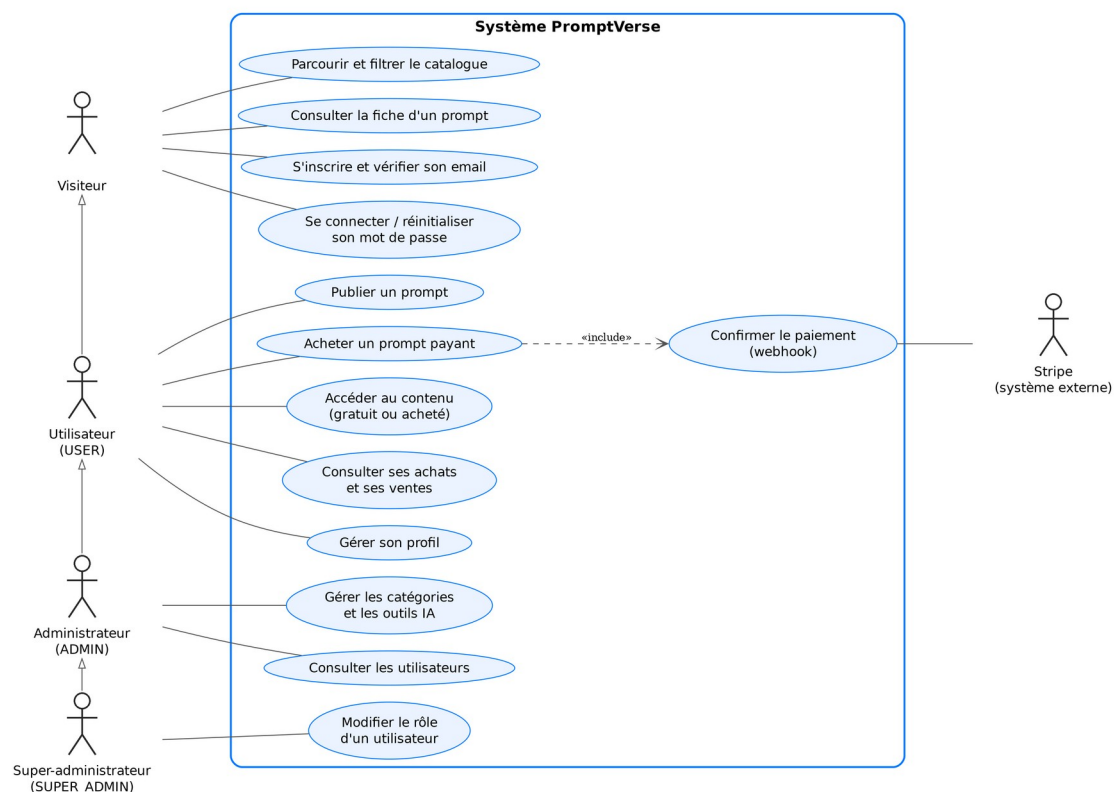


Figure 2 — Diagramme UML des cas d'utilisation

Les flèches à tête creuse entre les acteurs représentent l'héritage : un utilisateur peut faire tout ce que fait un visiteur, un administrateur tout ce que fait un utilisateur, et ainsi de suite. L'achat d'un prompt payant inclut la confirmation du paiement par Stripe.

5.3 Règles de gestion

Prompts et catalogue

- Un prompt est publié immédiatement (statut **PUBLISHED**) ; le statut **ARCHIVED** permet de le retirer du catalogue.
- Un prompt appartient à une seule catégorie et cible un seul outil IA ; il possède un titre, un slug unique, un prix, un aperçu public (**previewResult**), un contenu complet (**promptContent**), une image de couverture et des images d'aperçu facultatives.
- Le prix est compris entre 0 et 99,99 €. Un prompt à 0 € est **gratuit**.
- Les listes du catalogue ne renvoient **jamais** le contenu complet des prompts.

Visibilité du contenu complet

Sur la fiche d'un prompt, le contenu complet est renvoyé uniquement si l'une de ces conditions est vraie : l'utilisateur est le vendeur, le prompt est gratuit, ou l'utilisateur possède un achat au statut **COMPLETED** pour ce prompt. Dans tous les autres cas, y compris pour un visiteur, **promptContent** vaut **null**.

Achat

- Seul un utilisateur connecté peut acheter un prompt.
- L'achat est refusé si le prompt appartient à l'acheteur, s'il est **gratuit** (aucune session Stripe n'est créée : son contenu est déjà accessible) ou s'il a déjà été acheté.
- Un achat est d'abord créé au statut **PENDING** ; si l'acheteur relance le paiement, la même ligne **PENDING** est réutilisée.
- Seul le webhook Stripe, dont la signature est vérifiée, peut faire passer un achat à **COMPLETED**. À ce moment, le nombre de ventes du prompt augmente de 1 et le solde du vendeur est crédité du montant.
- Un même événement Stripe reçu plusieurs fois ne crédite le vendeur qu'une seule fois (**idempotence**). Une session expirée fait passer l'achat à **FAILED**.
- Le montant et le vendeur sont copiés dans l'achat : un changement ultérieur du prix du prompt ne modifie pas l'historique.

5.4 Maquettes

L'interface a été conçue en trois étapes : un **wireframe** basse fidélité pour organiser les zones de chaque écran, puis une **maquette haute fidélité** pour définir l'identité visuelle, et

enfin le développement. La charte retenue est un thème sombre, une couleur d'accent bleue (#0077FF) et la police Inter.

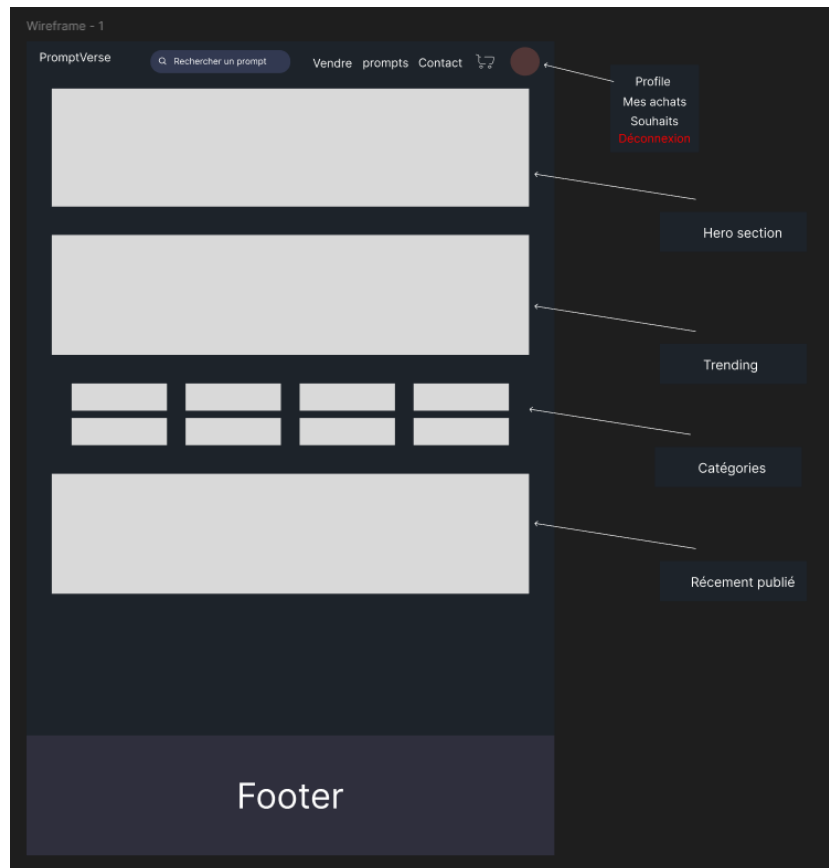


Figure 3 — Wireframe de la page d'accueil (basse fidélité)

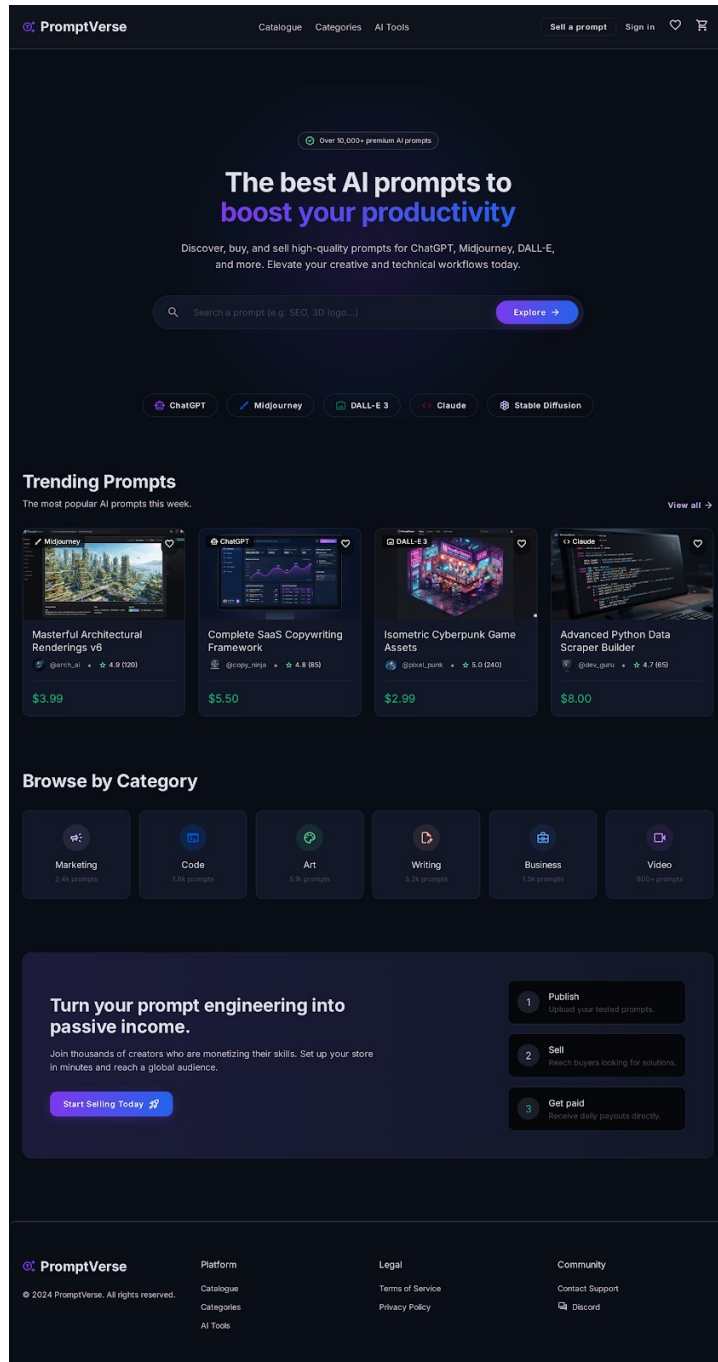


Figure 4 — Maquette haute fidélité de la page d'accueil

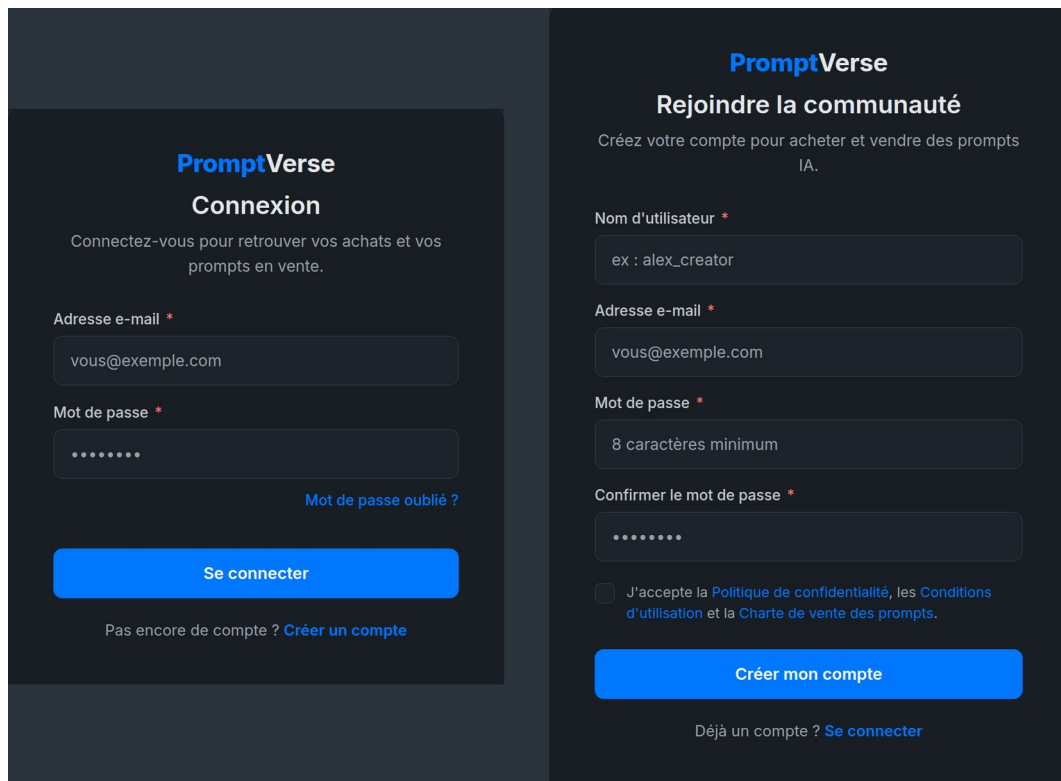


Figure 5 — Maquettes des écrans de connexion et d'inscription

La comparaison avec les captures de l'application réelle (section 8.3) montre que la structure prévue a été respectée ; certains contenus ont été simplifiés pour correspondre au périmètre livré (le panier et la liste de souhaits du wireframe ont par exemple été repoussés dans les évolutions).

5.5 Enchaînement des écrans

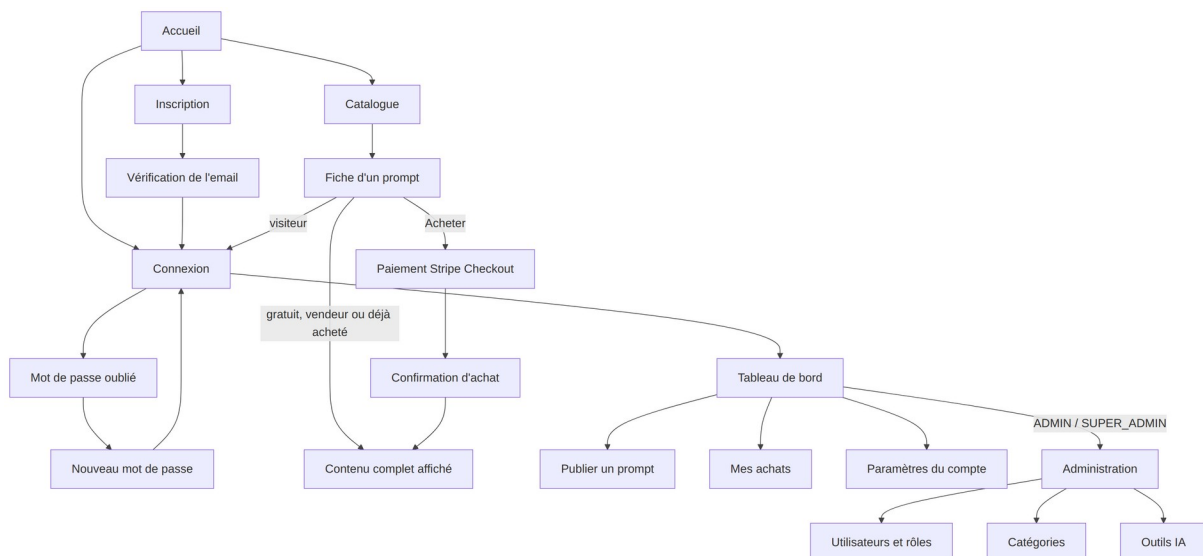


Figure 6 — Enchaînement des écrans de l'application

5.6 Diagrammes de séquence

Les deux scénarios les plus sensibles ont été décrits en diagrammes de séquence avant le développement. Ils m'ont permis d'identifier tôt les cas d'erreur à traiter : email non vérifié, jeton invalide, achat déjà effectué, webhook rejoué.

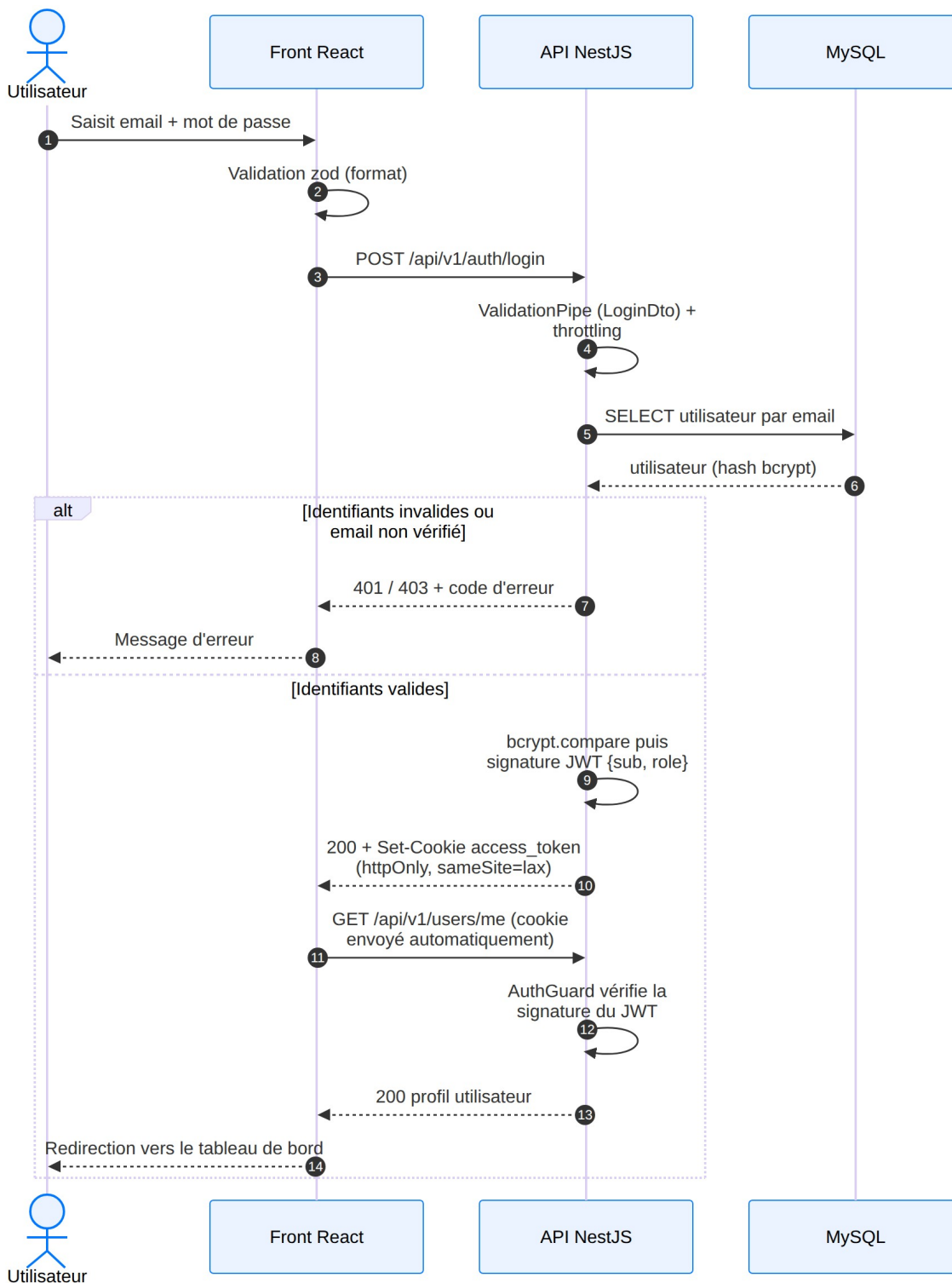


Figure 7 — Séquence de connexion et d'accès à une route protégée

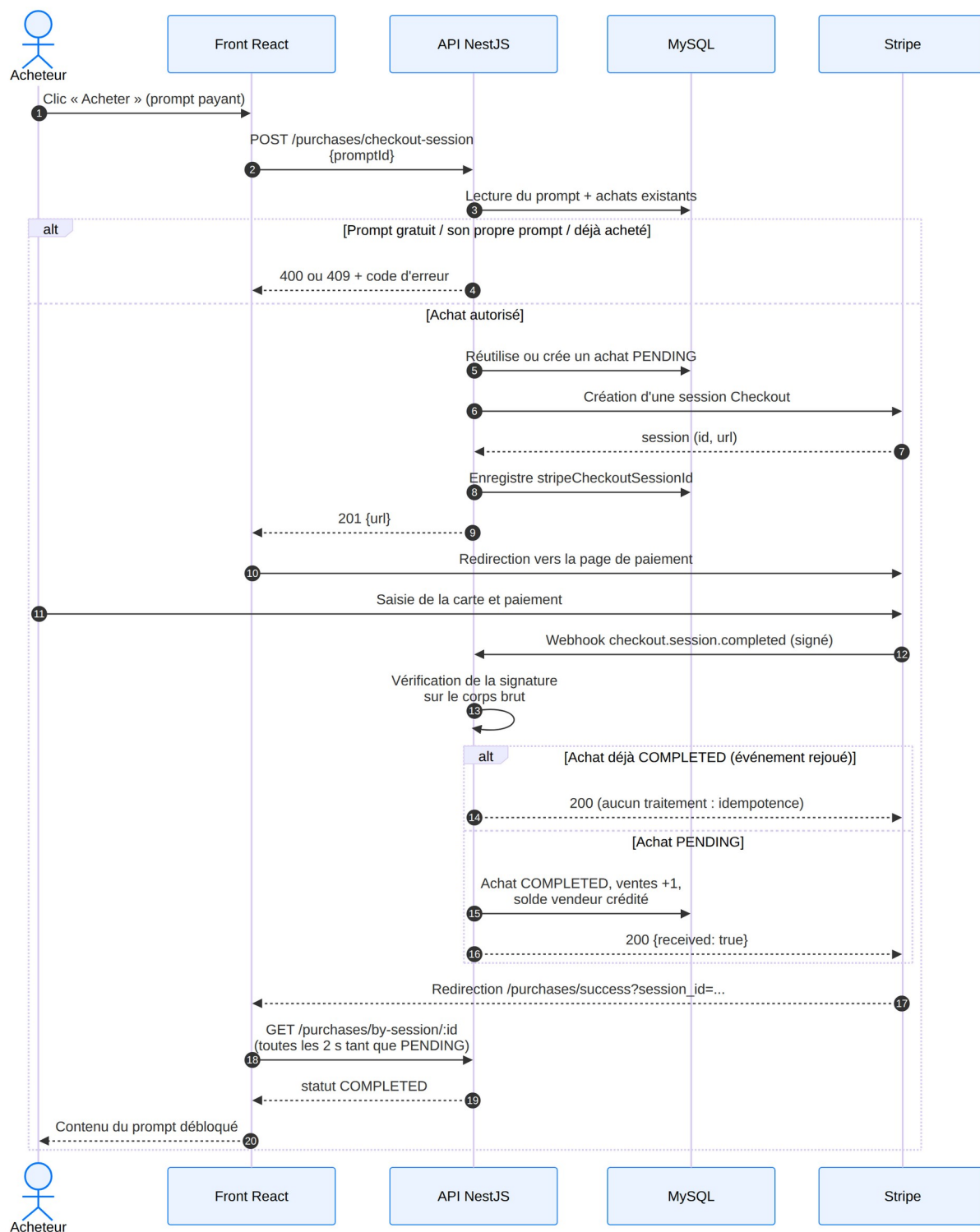


Figure 8 — Séquence d'achat d'un prompt payant avec Stripe

6. Spécifications techniques

6.1 Choix technologiques

Besoin	Choix	Justification
API back-end	NestJS 11 (TypeScript)	Imposé par le brief. Architecture modulaire (modules, contrôleurs, services), injection de dépendances, guards et pipes adaptés à la sécurité.
Accès aux données	TypeORM + MySQL 8.4	Base relationnelle adaptée à des données fortement liées (utilisateurs, prompts, achats) ; migrations versionnées ; requêtes paramétrées.
Front-end	React 19 + Vite + TypeScript	Imposé par le brief. Composants réutilisables, typage partagé avec l'API, build rapide.
Données serveur côté client	TanStack Query	Cache, invalidation et rechargement automatiques ; évite de recoder la gestion du chargement et des erreurs.
État client	Zustand	Store minimal pour la session et les notifications, sans la complexité de Redux.
Formulaires	react-hook-form + zod	Validation déclarative et typée, messages d'erreur immédiats (la validation serveur reste la référence).
Authentification	JWT + cookie httpOnly + bcrypt	Sans dépendance à Passport ni OAuth ; cookie inaccessible au JavaScript, ce qui limite l'impact d'une faille XSS.
Paieement	Stripe Checkout	Page de paiement hébergée et conforme PCI-DSS : aucune donnée bancaire ne passe par l'application.
Cache	Redis 7	Base NoSQL clé-valeur en mémoire, intégrée au module de cache de NestJS.
Fichiers	Multer + stockage disque	Solution la plus simple pour quelques images ; volume Docker persistant en production (MinIO, puis SeaweedFS, ont été testés puis abandonnés car surdimensionnés pour ce projet).
Industrialisation	Docker, GitHub Actions, Nginx	Environnements identiques du poste de développement à la production ; déploiement automatisé.

6.2 Architecture logicielle

PromptVerse suit une **architecture client / serveur en couches**. Le navigateur ne communique jamais directement avec la base de données : toutes les requêtes passent par l'API, qui centralise les règles métier et les contrôles de sécurité. En production, Nginx est le seul point d'entrée : il sert l'application React et redirige les appels `/api/` vers NestJS.

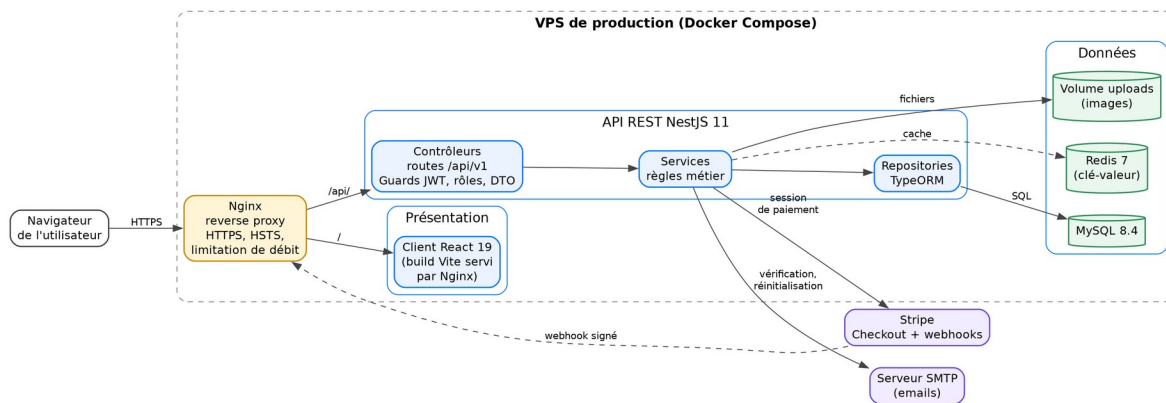


Figure 9 — Architecture applicative et de production

Organisation de l'API

Chaque domaine métier (auth, users, prompts, purchases, categories, ai-tools, admin, storage, mail, health) est un module NestJS isolé. À l'intérieur d'un module, les responsabilités sont séparées en couches :

Couche	Rôle	Exemple
Contrôleur	Expose les routes HTTP, applique les guards et les DTO, délègue au service	<code>PurchasesController</code>
DTO	Décrit et valide la forme des données entrantes	<code>CreateCheckoutSessionDto</code> (<code>promptId</code> doit être un UUID)
Service	Porte les règles métier et orchestre les accès aux données	<code>PurchasesService.createCheckoutSession()</code>
Repository	Accède à la base via TypeORM	<code>Repository<Purchase></code>
Entité	Décrit la table et ses relations	<code>Purchase</code>
Transverse	Guards, décorateurs, codes d'erreur, validation de la configuration	<code>AuthGuard</code> , <code>@Roles()</code> , <code>ErrorCodes</code>

Toutes les routes sont préfixées par `/api` et versionnées dans l'URL (`/api/v1/...`), ce qui permettra de faire évoluer l'API sans casser les clients existants. L'API est documentée avec Swagger, accessible sur `/api/docs` en développement.

Organisation du front-end

- `pages/` : un composant par écran (accueil, catalogue, fiche, authentification, tableau de bord, administration).
- `components/` : composants réutilisables regroupés par domaine (layout, auth, prompt, admin, ui).
- `common/api/` : un fichier par ressource d'API, basé sur une instance Axios unique qui envoie le cookie de session.
- `hooks/` : requêtes et mutations TanStack Query (`usePrompts`, `usePurchases...`).
- `common/store/` : stores Zustand (session, notifications, interface).

6.3 Environnement de développement

L'environnement complet est décrit dans un fichier `docker-compose.yml`. Une seule commande démarre tous les services, avec des contrôles de santé qui garantissent que l'API ne démarre qu'une fois la base de données prête.

Service	Image / technologie	Rôle
api	NestJS (Node.js 22)	API REST, construite à partir de son Dockerfile
client	React + Vite	Application web
database	MySQL 8.4	Base de données relationnelle
redis	Redis 7	Cache clé-valeur
mailpit	Mailpit	Capture les emails (vérification, réinitialisation) sans les envoyer réellement
redisinsight	RedisInsight	Interface d'inspection du cache

La configuration passe par des variables d'environnement, validées au démarrage par un schéma **Joi** : si une variable obligatoire manque (secret JWT, URL de la base, clés Stripe...), l'API refuse de démarrer au lieu de planter plus tard. Des fichiers d'exemple (`.env.sample`, `.env.example`) sont versionnés, mais jamais les fichiers contenant les vraies valeurs.

6.4 Sécurité de l'application

Mesure	Mise en œuvre	Risque couvert
Mots de passe	Hachage bcrypt (coût 10), champ exclu de toutes les réponses (<code>@Exclude</code>)	Vol de base de données, fuite de données
Authentification	JWT signé contenant l'identifiant et le rôle, stocké dans un cookie httpOnly, <code>secure</code> en production, <code>sameSite=lax</code> , durée 24 h	Vol de session, XSS, CSRF
Protection par défaut	Guard global : toute route est privée sauf si elle est marquée <code>@Public()</code>	Oubli de protection d'une route
Contrôle d'accès	<code>AuthRolesGuard</code> + décorateur <code>@Roles()</code> sur l'administration	Élévation de privilèges
Validation	<code>ValidationPipe</code> global avec <code>whitelist</code> et <code>forbidNonWhitelisted</code>	Injection de champs, données malformées
Limitation de débit	Throttler global (par utilisateur ou par IP) + limites renforcées sur l'inscription et la connexion ; limitation aussi au niveau de Nginx	Force brute, déni de service
Injection SQL	Requêtes paramétrées générées par TypeORM	Injection SQL
En-têtes HTTP	Helmet côté API ; HSTS, X-Frame-Options, X-Content-Type-Options côté Nginx	Clickjacking, sniffing MIME
CORS	Origine limitée à l'URL du client, cookies autorisés uniquement pour elle	Appels depuis un site tiers
Webhook Stripe	Vérification de la signature sur le corps brut, route exclue du throttling	Fausse confirmation de paiement

Mesure	Mise en œuvre	Risque couvert
Contenu payant	Colonne exclue des listes, renvoyée seulement si l'accès est autorisé	Accès gratuit à un contenu payant
Fichiers envoyés	Type image uniquement, taille maximale, nom de fichier régénéré (UUID), dossiers autorisés en liste blanche	Écrasement de fichiers, parcours de répertoires
Transport	HTTPS obligatoire (redirection 301), certificat Let's Encrypt	Interception réseau

6.5 Données personnelles et accessibilité

RGPD. L'application ne collecte que les données nécessaires au service : adresse email, nom d'utilisateur, mot de passe haché et, facultativement, un avatar, une biographie et des liens sociaux. Aucune donnée bancaire n'est stockée : Stripe s'en charge. L'inscription demande l'acceptation des conditions d'utilisation et de la politique de confidentialité. L'utilisateur peut modifier ses informations et supprimer son compte depuis ses paramètres. La suppression est aujourd'hui une suppression logique (`deletedAt`), nécessaire pour conserver l'historique des ventes ; une anonymisation des données personnelles lors de la suppression fait partie des améliorations prévues (section 13.2).

Accessibilité. Les formulaires associent chaque champ à un libellé (`label` / `htmlFor`), les images portent un texte alternatif, les messages d'erreur sont affichés en texte sous le champ concerné et le thème sombre respecte un contraste élevé entre le texte et le fond. Un audit complet selon le RGAA reste à mener.

7. Conception de la base de données

La base de données a été conçue avec la méthode **Merise** avant l'implémentation : dictionnaire de données, modèle conceptuel (MCD), modèle logique (MLD) puis modèle physique (MPD). Le modèle a ensuite été traduit en entités TypeORM et en migrations.

7.1 Dictionnaire de données (extrait)

Entité	Donnée	Type	Contraintes
Utilisateur	email	VARCHAR(255)	obligatoire, unique
	username	VARCHAR(50)	obligatoire, unique
	password	VARCHAR(255)	obligatoire, haché avec bcrypt
	balance	DECIMAL(10,2)	défaut 0
	role / status	ENUM	USER, ADMIN, SUPER_ADMIN / INACTIVE, ACTIVE, BANNED
	isEmailVerified	BOOLEAN	défaut faux
Prompt	title	VARCHAR(255)	obligatoire, 3 à 255 caractères
	slug	VARCHAR(255)	obligatoire, unique
	promptContent	TEXT	obligatoire, jamais exposé dans les listes
	price	DECIMAL(10,2)	0 à 99,99
	salesCount	INT	défaut 0
	status	ENUM	PUBLISHED, ARCHIVED
Catégorie / Outil IA	name, slug	VARCHAR(255)	obligatoires, uniques ; suppression logique
Achat	amount	DECIMAL(10,2)	copie du prix au moment de l'achat
	stripeCheckoutSessionId	VARCHAR(255)	obligatoire, unique
	status	ENUM	PENDING, COMPLETED, FAILED

Les identifiants sont des **UUID** plutôt que des entiers auto-incrémentés : ils ne sont pas devinables et ne révèlent pas le volume de données. Les montants utilisent le type **DECIMAL** pour éviter les erreurs d'arrondi des nombres flottants.

7.2 Modèle conceptuel de données (MCD)

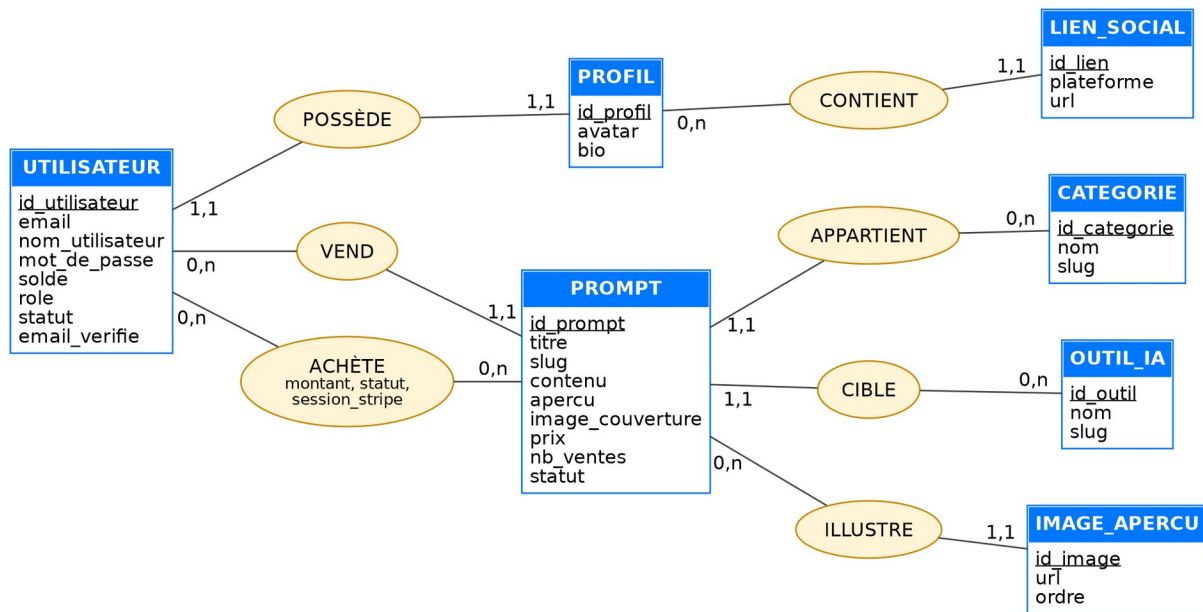


Figure 10 — MCD de PromptVerse

L'association **ACHÈTE** porte ses propres propriétés (montant, statut, identifiant de session Stripe) : un utilisateur peut acheter plusieurs prompts et un prompt peut être acheté par plusieurs utilisateurs. Au passage au modèle logique, elle devient donc une table à part entière.

7.3 Modèle logique de données (MLD)

Notation : clé primaire soulignée dans le MCD, précédée de # ici ; clé étrangère suivie de *.

```

USERS (#id, email, username, password, balance, role, status, isEmailVerified, deletedAt)
USER_PROFILES (#id, avatar, bio, user_id*)
SOCIAL_LINKS (#id, platform, url, profile_id*)
CATEGORIES (#id, name, slug, iconUrl, deletedAt)
AI_TOOLS (#id, name, slug, iconUrl, deletedAt)
PROMPTS (#id, title, slug, promptContent, previewResult, coverImage, price, salesCount,
          status, sellerId*, categoryId*, aiToolId*)
PREVIEW_IMAGES (#id, url, sortOrder, promptId*)
PURCHASES (#id, amount, stripeCheckoutSessionId, stripePaymentIntentId, status,
            buyerId*, promptId*, sellerId*)
  
```

Les règles de suppression ont été choisies selon la nature des données : **CASCADE** pour les données dépendantes (profil, liens sociaux, images d'aperçu), **RESTRICT** pour les prompts et les achats, afin de ne jamais perdre l'historique des ventes.

7.4 Modèle physique de données (MPD)

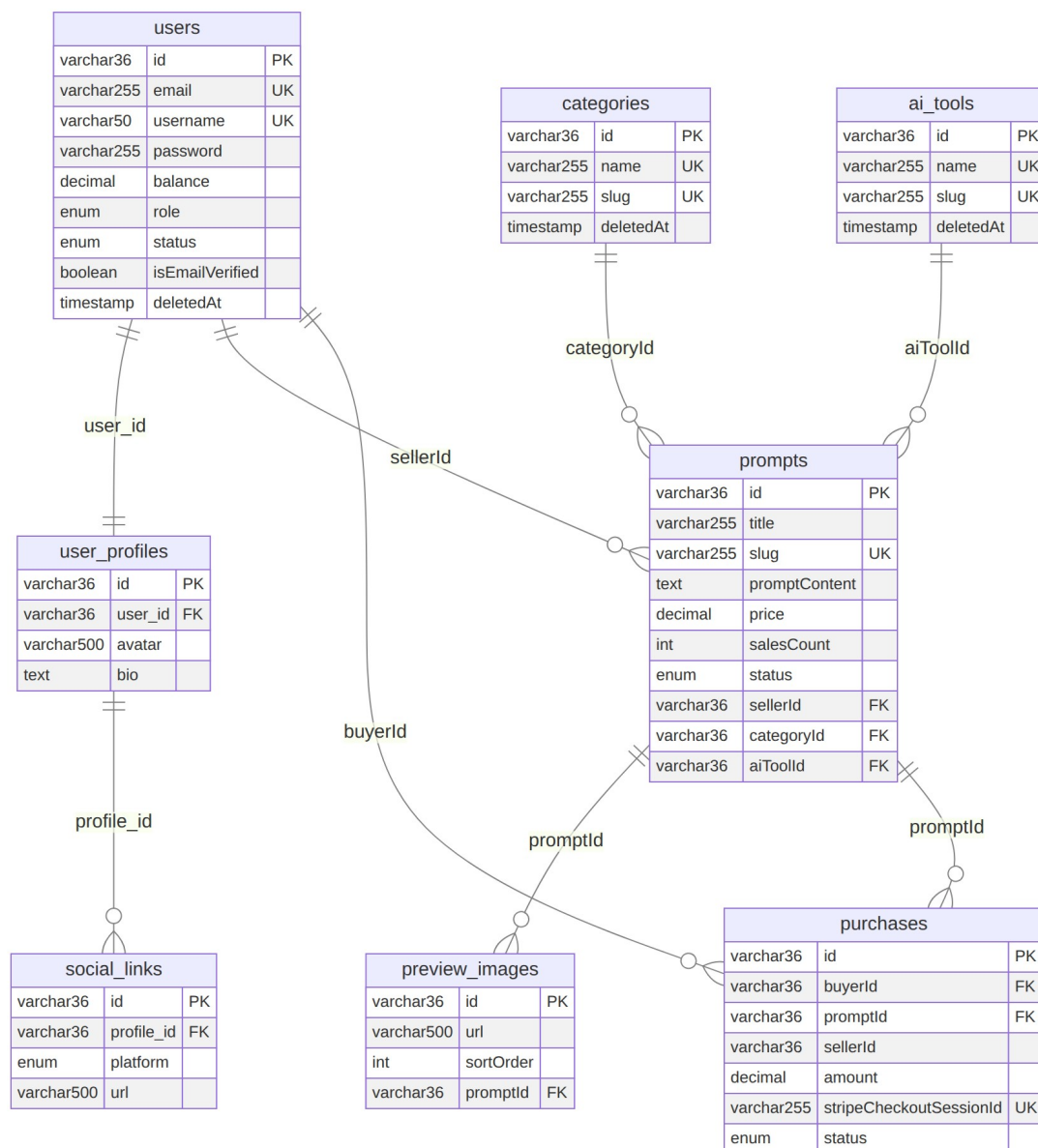


Figure 11 — Modèle physique des tables MySQL

Le schéma est créé et modifié uniquement par des **migrations TypeORM** versionnées (une par domaine : utilisateurs, outils IA, catégories, prompts, achats). En production, elles sont exécutées automatiquement au démarrage du conteneur de l'API. Extrait de la migration de la table des achats :

```

await queryRunner.query(`CREATE TABLE \`${purchases}\` (
  \`${id}\` varchar(36) NOT NULL,
  \`${buyerId}\` varchar(255) NOT NULL, \`${promptId}\` varchar(255) NOT NULL,
  \`${sellerId}\` varchar(255) NOT NULL,
  \`${amount}\` decimal(10,2) NOT NULL,
  \`${stripeCheckoutSessionId}\` varchar(255) NOT NULL,
  \`${status}\` enum ('PENDING', 'COMPLETED', 'FAILED') NOT NULL DEFAULT 'PENDING',
  ...
)`);
await queryRunner.query(`ALTER TABLE \`${purchases}\` ADD CONSTRAINT
\`${FK_22533450a1d627dc3ae071f22ae}\`
  FOREIGN KEY (\`${buyerId}\`) REFERENCES \`${users}\`(\`${id}\`) ON DELETE RESTRICT`);

```

7.5 Diagramme de classes

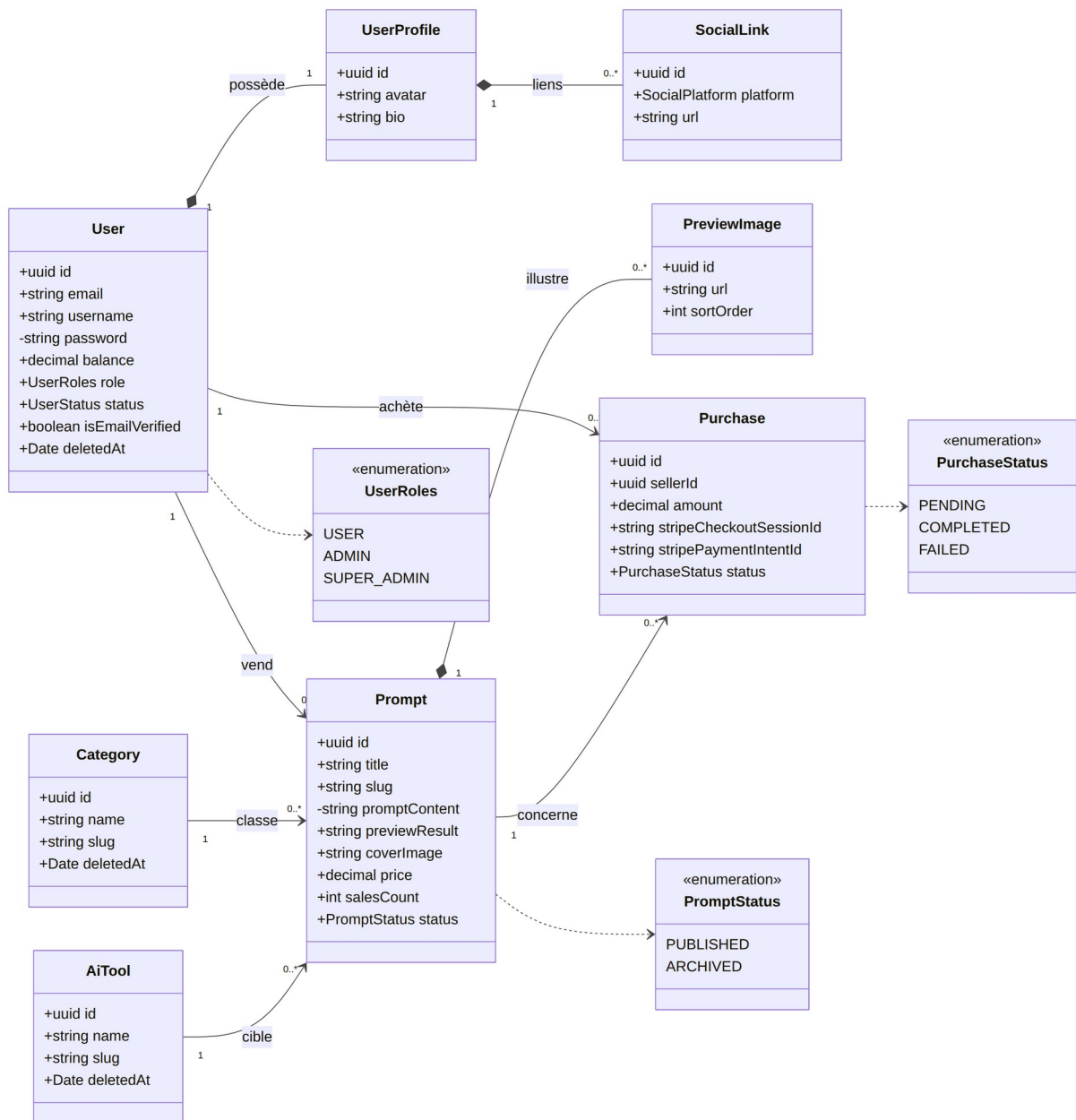


Figure 12 — Diagramme de classes des entités métier

7.6 Composants d'accès aux données SQL et NoSQL

Accès SQL avec TypeORM

Les services n'écrivent jamais de SQL à la main : ils utilisent des repositories TypeORM injectés par NestJS. Les requêtes générées sont paramétrées, ce qui protège contre l'injection SQL. Pour les listes publiques, j'ai défini une **sélection explicite des colonnes** : le contenu payant n'est même pas lu en base, ce qui allège les réponses et empêche toute fuite.

apps/api/src/modules/prompts/prompts.service.ts (extrait simplifié)

```
// promptContent (le texte payant) est volontairement exclu des vues liste
private static readonly LIST_SELECT: FindOptionsSelect<Prompt> = {
  id: true, title: true, slug: true, previewResult: true, coverImage: true,
  price: true, salesCount: true, status: true, createdAt: true,
  sellerId: true, categoryId: true, aiToolId: true,
};

findAll(pageNumber: number, promptsPerPage: number) {
  return this.promptsRepository.find({
    select: PromptsService.LIST_SELECT,
    where: { status: PromptStatus.PUBLISHED },
    skip: promptsPerPage * (pageNumber - 1),
    take: promptsPerPage,
    order: { createdAt: 'DESC' },
  });
}

incrementSalesCount(id: string) {
  // UPDATE atomique côté base : pas de lecture puis écriture concurrente
  return this.promptsRepository.increment({ id }, 'salesCount', 1);
}
```

Accès NoSQL avec Redis

Redis est une base de données **NoSQL de type clé-valeur**, stockée en mémoire. Je l'ai intégré au module de cache de NestJS : le store est configuré une seule fois, globalement, et les services y accèdent par injection du **CACHE_MANAGER** avec des opérations **get**, **set** et **del** sur des clés, avec une durée de vie par défaut de 10 minutes.

apps/api/src/app.module.ts (extrait)

```
CacheModule.registerAsync({
  isGlobal: true,
  inject: [ConfigService],
  useFactory: (config: ConfigService) => ({
    ttl: 10 * 60 * 1000, // 10 minutes
    stores: [createKeyv(config.getOrThrow<string>('REDIS_URL'))],
  }),
});
```

Point d'honnêteté. L'infrastructure Redis est en place et fonctionnelle (connexion validée au démarrage, service présent en production), mais l'application ne met pas encore réellement de lectures en cache : le seul appel actuel supprime une clé. Mettre en cache les lectures publiques les plus fréquentes (catégories, outils IA, première page du catalogue) avec une invalidation lors des modifications est la première amélioration prévue (section 13.2).

8. Réalisations

Cette section présente les extraits de code les plus significatifs. Ils ont été choisis parce qu'ils portent une règle métier importante ou une mesure de sécurité.

8.1 Back-end

Authentification par cookie `httpOnly`

À la connexion, après vérification du mot de passe et de l'email, l'API signe un jeton qui contient seulement l'identifiant et le rôle. Elle ne le renvoie pas dans le corps de la réponse : elle le place dans un cookie **httpOnly**, que le JavaScript du navigateur ne peut pas lire.

apps/api/src/modules/auth/auth.controller.ts (extrait simplifié)

```
@Post('login')
@Throttle({
  short: { limit: 2, ttl: 1000 },
  medium: { limit: 5, ttl: 60000, blockDuration: 15 * 60 * 1000 }, // 15 min de blocage
})
@Public()
@HttpCode(HttpStatus.OK)
async login(@Body() loginDto: LoginDto, @Res({ passthrough: true }) res: Response) {
  const { accessToken } = await this.authService.login(loginDto);

  res.cookie('access_token', accessToken, {
    httpOnly: true, // inaccessible au JavaScript
    secure: process.env.NODE_ENV === 'production', // HTTPS uniquement en production
    sameSite: 'lax', // limite les requêtes cross-site
    maxAge: 1000 * 60 * 60 * 24, // 24 heures
  });
  return { message: 'Login successful' };
}
```

Guard global : toutes les routes sont protégées par défaut

Le guard est enregistré globalement (`APP_GUARD`). Plutôt que d'ajouter une protection sur chaque route privée — avec le risque d'en oublier une — c'est l'inverse : une route est privée sauf si elle est explicitement marquée `@Public()`.

apps/api/src/common/guards/auth.guard.ts (extrait)

```
async canActivate(context: ExecutionContext): Promise<boolean> {
  const isPublic = this.reflector.getAllAndOverride<boolean>(IS_PUBLIC_KEY, [
    context.getHandler(), context.getClass(),
  ]);
  if (isPublic) return true;

  const request: ExtendedRequest = context.switchToHttp().getRequest();
  const token = request.cookies?.access_token;
  if (!token) {
    if (isOptionalAuth) return true;          // ex. fiche d'un prompt : visiteur accepté
    throw new UnauthorizedException({ code: ErrorCodes.NO_TOKEN_PROVIDED });
  }
  try {
    request[CURRENT_USER_KEY] = await this.jwtService.verifyAsync<jwtPayloadType>(token, {
      secret: this.configService.getOrThrow<string>('JWT_SECRET'),
    });
    return true;
  } catch {
    if (isOptionalAuth) return true;
    throw new UnauthorizedException({ code: ErrorCodes.INVALID_TOKEN });
  }
}
```

Contrôle d'accès par rôles

apps/api/src/modules/admin/controllers/admin-users.controller.ts (extrait)

```
@UseGuards(AuthRolesGuard)
@Roles(UserRoles.ADMIN, UserRoles.SUPER_ADMIN)          // tout le contrôleur :
administrateurs
@Controller({ path: 'admin/users', version: '1' })
export class AdminUsersController {
  @Get()
  findAll() { return this.adminUsersService.findAll(); }

  @Roles(UserRoles.SUPER_ADMIN)                          // cette route : super-admin
  uniquement
  @Patch(':userId/make-role')
  makeRole(@Param('userId') userId: string, @Body() dto: UpdateRoleDto) {
    return this.adminUsersService.makeRole(userId, dto.role);
  }
}
```

Masquage du contenu payant et prompts gratuits

apps/api/src/modules/prompts/prompts.service.ts (extrait)

```
async findOneBySlug(slug: string, currentUserId?: string) {
  const prompt = await this.promptsRepository.findOne({
    where: { slug: slug.trim().toLowerCase(), status: PromptStatus.PUBLISHED },
    relations: { previewImages: true },
  });
  if (!prompt) throw new NotFoundException({ code: ErrorCodes.PROMPT_NOT_FOUND });

  const isOwner = currentUserId === prompt.sellerId;
  const isFree = PromptsService.isFree(prompt.price);
  // La base n'est interrogée que si c'est nécessaire
  const hasPurchased = !isOwner && !isFree && currentUserId
    ? await this.hasCompletedPurchase(currentUserId, prompt.id)
    : false;
  const canViewFullContent = isOwner || isFree || hasPurchased;

  return {
    ...prompt,
    promptContent: canViewFullContent ? prompt.promptContent : null,
    isPurchasedByCurrentUser: hasPurchased,
  };
}
```

Création d'une session de paiement Stripe

Avant de contacter Stripe, le service applique toutes les règles métier. Un prompt gratuit est refusé : son contenu est déjà accessible, il n'y a donc rien à payer.

apps/api/src/modules/purchases/purchases.service.ts (extrait simplifié)

```
async createCheckoutSession(buyerId: string, promptId: string) {
  const prompt = await this.promptsService.findByIdForPurchase(promptId);

  if (PromptsService.isFree(prompt.price))
    throw new BadRequestException({ code: ErrorCodes.PROMPT_IS_FREE });
  if (prompt.sellerId === buyerId)
    throw new BadRequestException({ code: ErrorCodes.CANNOT_PURCHASE_OWN_PROMPT });
  if (await this.promptsService.hasCompletedPurchase(buyerId, promptId))
    throw new ConflictException({ code: ErrorCodes.PROMPT_ALREADY_PURCHASED });

  // Réutilise une tentative PENDING plutôt que d'empiler une ligne à chaque clic
  let purchase = await this.purchasesRepository.findOne({
    where: { buyerId, promptId, status: PurchaseStatus.PENDING },
  });
  purchase ??= await this.purchasesRepository.save(this.purchasesRepository.create({
    buyerId, promptId, sellerId: prompt.sellerId, amount: prompt.price,
    stripeCheckoutSessionId: `pending-${buyerId}-${promptId}-${Date.now()}`,
    status: PurchaseStatus.PENDING,
  }));

  const session = await this.stripeService.createCheckoutSession({
    mode: 'payment',
    line_items: [{ quantity: 1, price_data: {
      currency: 'eur', unit_amount: PurchasesService.eurosToCents(prompt.price),
      product_data: { name: prompt.title } } }],
    success_url: `${clientUrl}/purchases/success?session_id={CHECKOUT_SESSION_ID}`,
    cancel_url: `${clientUrl}/prompts/${prompt.slug}`,
    metadata: { buyerId, promptId, purchaseId: purchase.id },
  });

  purchase.stripeCheckoutSessionId = session.id;
  await this.purchasesRepository.save(purchase);
  return { url: session.url };
}
```

Webhook Stripe signé et idempotent

Stripe peut envoyer plusieurs fois le même événement. Le traitement vérifie donc l'état de l'achat avant de le modifier : un achat déjà **COMPLETED** n'est pas traité une seconde fois, et le vendeur n'est crédité qu'une fois.

purchases.controller.ts et purchases.service.ts (extraits)

```
@Post('webhook')
@Public() // Stripe ne possède pas de cookie : la sécurité repose sur la signature
@SkipThrottle() // Stripe ne doit jamais recevoir d'erreur 429
@HttpCode(HttpStatus.OK)
async handleWebhook(@Req() req: RawBodyRequest<Request>,
  @Headers('stripe-signature') signature: string | undefined) {
  if (!req.rawBody || !signature) throw new BadRequestException('Missing Stripe signature or body');
  await this.purchasesService.handleWebhookEvent(req.rawBody, signature);
  return { received: true };
}

private async completePurchase(session: Stripe.Checkout.Session) {
  const purchase = await this.purchasesRepository.findOne({
    where: { stripeCheckoutSessionId: session.id },
  });
  if (!purchase) return;
  if (purchase.status === PurchaseStatus.COMPLETED) return; // idempotence

  purchase.status = PurchaseStatus.COMPLETED;
}
```

```

    await this.purchasesRepository.save(purchase);
    await this.promptsService.incrementSalesCount(purchase.promptId);

    const seller = await this.userService.findById(purchase.sellerId);
    if (seller) {
      seller.balance = Number(seller.balance) + Number(purchase.amount);
      await this.userService.save(seller);
    }
  }
}

```

Envoi sécurisé des images

apps/api/src/modules/prompts/prompts.module.ts (extrait)

```

MulterModule.register({
  storage: diskStorage({
    destination: (req, file, cb) => {
      const folder = file.fieldname === 'coverImage'
        ? StorageFolder.PROMPT_COVERS : StorageFolder.PROMPT_PREVIEWS;
      cb(null, `./uploads/${folder}`);
    },
    // Le nom d'origine n'est jamais réutilisé : impossible d'écraser un fichier existant
    filename: (req, file, cb) => cb(null, `${randomUUID()}${extname(file.originalname)}`),
  }),
  fileFilter: (req, file, cb) => file.mimetype.startsWith('image')
    ? cb(null, true)
    : cb(new BadRequestException({ code: ErrorCodes.UNSUPPORTED_FILE_TYPE }), false),
  limits: { fileSize: 1024 * 1024 * 8 }, // 8 Mo
}),

```

Les fichiers sont ensuite servis par un contrôleur qui n'accepte que les dossiers d'une liste blanche et utilise `res.sendFile` avec un dossier racine, ce qui empêche de remonter dans l'arborescence du serveur (`../`). La section 10.4 présente une limite de ce filtre identifiée lors de ma veille.

8.2 Front-end

Routes protégées

Les routes privées sont regroupées sous un composant `ProtectedRoute`. Pendant la vérification de la session, un indicateur de chargement s'affiche ; un visiteur est redirigé vers la connexion, en mémorisant la page demandée pour l'y renvoyer ensuite. Cette protection améliore l'expérience utilisateur, mais la vraie sécurité reste assurée par l'API.

apps/client/src/components/auth/ProtectedRoute.tsx et routes.tsx (extraits)

```
export function ProtectedRoute() {
  const { status } = useAuth();
  const location = useLocation();

  if (status === "pending") {
    return <PageLoader label="Vérification de la session..." />;
  }
  if (status === "unauthenticated") {
    return <Navigate to="/login" replace
      state={{ from: location.pathname + location.search }} />;
  }
  return <Outlet />;
}

// routes.tsx
<Route element={<ProtectedRoute />}>
  <Route path="/dashboard" element={<DashboardLayout />} ... </Route>
  <Route element={<AdminRoute />}>
    <Route path="/admin" element={<AdminLayout />} ... </Route>
  </Route>
</Route>
```

Client HTTP et session

apps/client/src/common/api/client.ts (extrait)

```
export const api = axios.create({
  baseURL,
  withCredentials: true, // indispensable : le navigateur renvoie le cookie httpOnly
  headers: { "Content-Type": "application/json" },
});

// Une 401 signifie que la session n'est plus valable : on réinitialise le store,
// ce qui redirige automatiquement les routes protégées vers /login.
api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error?.response?.status === 401) useAuthStore.getState().reset();
    return Promise.reject(error);
  },
);
```

Formulaire d'inscription validé avec zod

apps/client/src/pages/auth/RegisterPage.tsx (extrait)

```
const schema = z.object({
  username: z.string().min(3, "3 caractères minimum.").max(50, "50 caractères maximum."),
  email: z.string().min(1, "L'e-mail est requis.").email("E-mail invalide."),
  password: z.string().min(6, "6 caractères minimum."),
  confirmPassword: z.string().min(1, "Confirmez le mot de passe."),
  terms: z.boolean().refine((v) => v === true, "Vous devez accepter les conditions."),
}).refine((data) => data.password === data.confirmPassword, {
  path: ["confirmPassword"],
  message: "Les mots de passe ne correspondent pas.",
});

const { register, handleSubmit, formState } = useForm<FormValues>({
  resolver: zodResolver(schema),
});
```

Achat et attente de la confirmation du paiement

Le paiement est confirmé de façon asynchrone par le webhook : quand l'acheteur revient de Stripe, l'achat peut encore être **PENDING** pendant quelques secondes. La page de confirmation interroge donc l'API toutes les 2 secondes, et s'arrête dès que le statut change.

PromptDetailPage.tsx et hooks/usePurchases.ts (extraits)

```
async function handleBuyClick() {
  if (!isAuthenticated) {
    navigate("/login", { state: { from: location.pathname } });
    return;
  }
  try {
    const { url } = await createCheckoutSession.mutateAsync(promptId);
    window.location.href = url; // redirection vers Stripe Checkout
  } catch (err) {
    toast.error(getApiErrorMessage(err)); // ex. « Ce prompt est gratuit »
  }
}

export function usePurchaseBySessionId(sessionId: string | undefined) {
  return useQuery({
    queryKey: queryKeys.purchases.bySession(sessionId ?? ""),
    queryFn: () => purchasesApi.getBySessionId(sessionId as string),
    enabled: !!sessionId,
    refetchInterval: (query) =>
      (query.state.data as Purchase | undefined)?.status === "PENDING" ? 2000 : false,
  });
}
```

8.3 Captures d'écran de l'application

Les captures suivantes ont été réalisées sur l'application lancée en local avec Docker Compose et des données de démonstration.

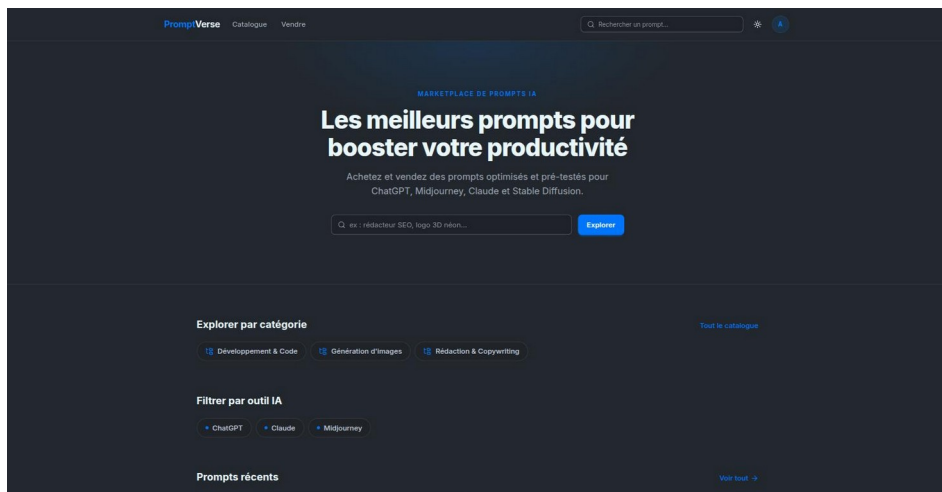


Figure 13 — Page d'accueil : catégories, outils IA et prompts récents

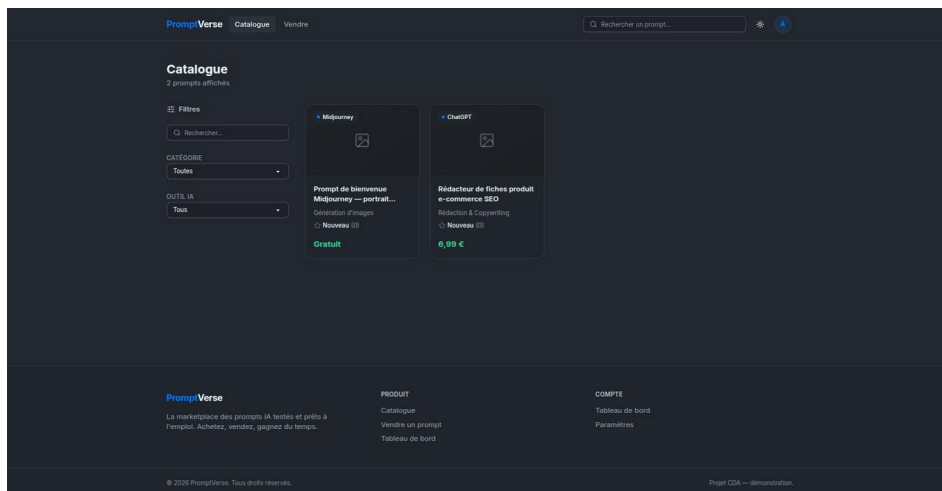


Figure 14 — Catalogue filtrable, avec badge « Gratuit » sur les prompts à 0 €

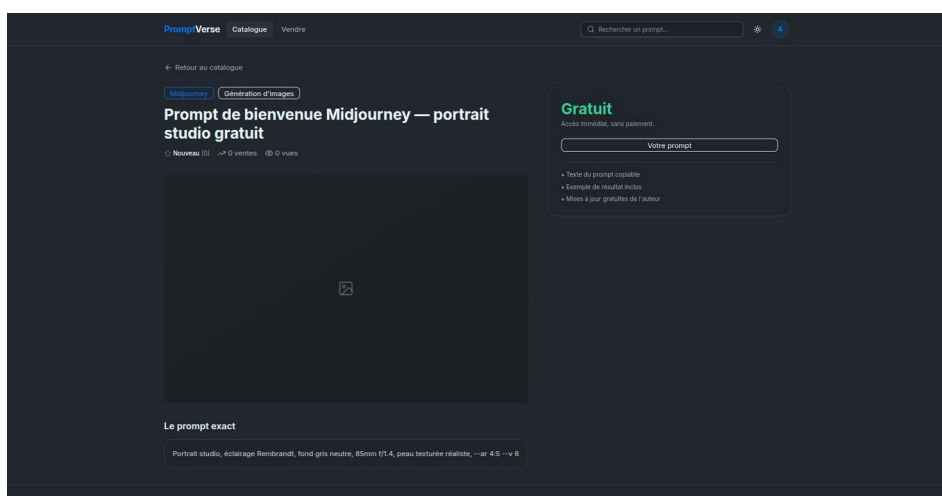


Figure 15 — Fiche d'un prompt gratuit : contenu accessible sans passer par Stripe

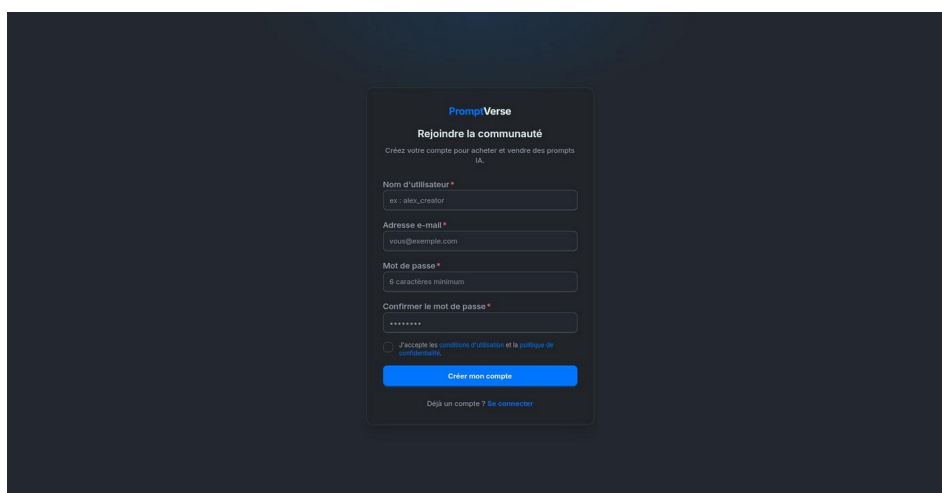


Figure 16 — Page d'inscription

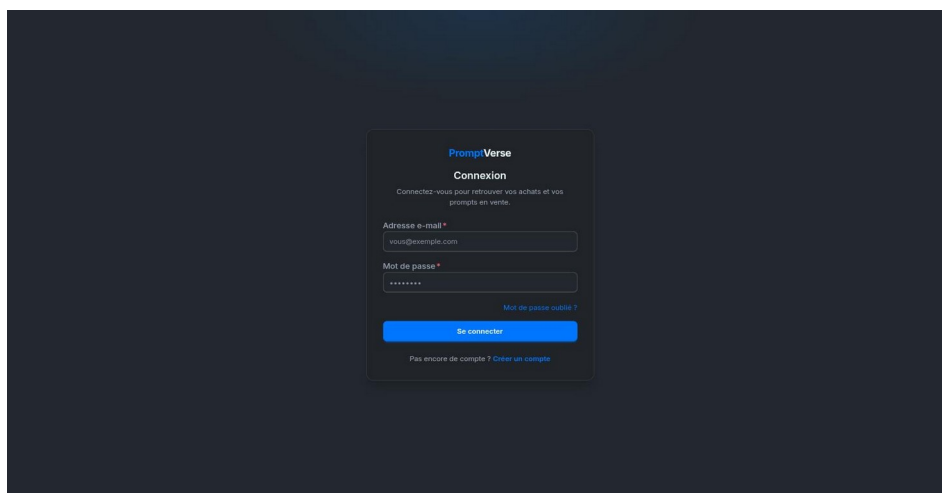


Figure 17 — Page de connexion

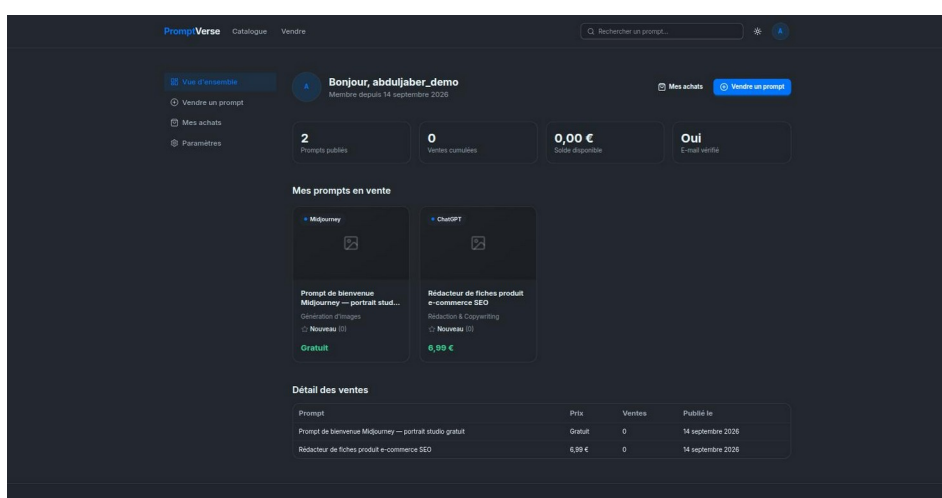


Figure 18 — Tableau de bord d'un vendeur : statistiques, prompts publiés et ventes

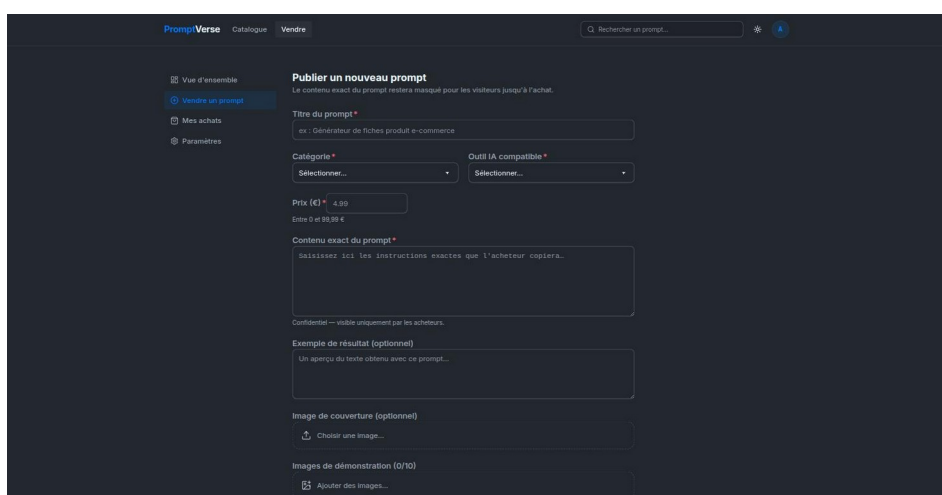


Figure 19 — Formulaire de publication d'un prompt

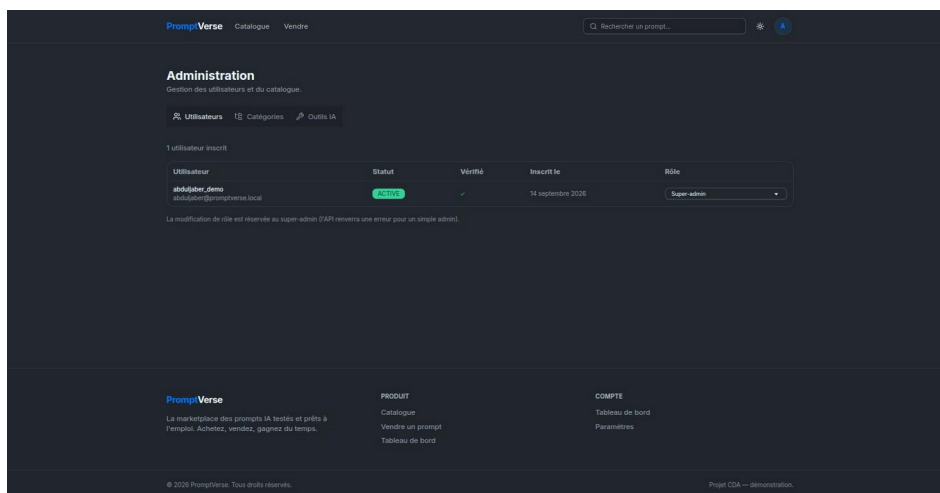


Figure 20 — Administration des utilisateurs : modification de rôle réservée au SUPER_ADMIN

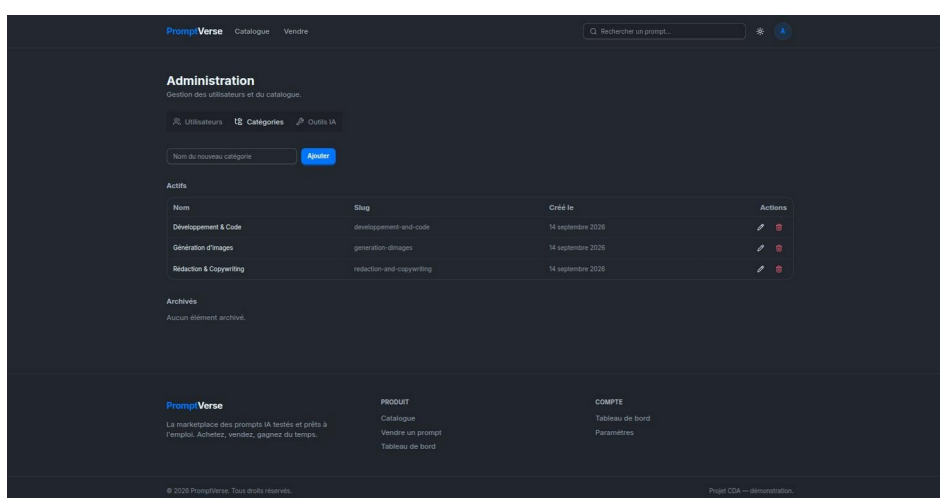


Figure 21 — Administration des catégories

9. Plan de tests et jeu d'essai

9.1 Stratégie de tests

Les tests ont été construits à partir des critères d'acceptation des user stories. J'ai combiné trois niveaux :

Niveau	Outil	Objectif	Exécution
Tests unitaires	Jest + mocks	Vérifier les règles métier des services, sans base de données ni Stripe	Local et automatiquement dans la CI
Tests d'API (jeu d'essai)	Script HTTP sur l'API lancée avec Docker	Vérifier le comportement réel de bout en bout : guards, validation, codes HTTP, base de données	Local, avant chaque livraison importante
Tests manuels	Navigateur, Swagger, Mailpit	Vérifier les parcours utilisateur et l'affichage	À la fin de chaque sprint
Contrôles statiques	ESLint, compilateur TypeScript, build Vite	Détecter les erreurs avant l'exécution	Automatiquement dans la CI

Pour les tests unitaires, les dépendances (repositories TypeORM, service Stripe, service utilisateurs) sont remplacées par des **mocks** : chaque test contrôle exactement ce que renvoient les dépendances et vérifie la réaction du service. Les tests sont rapides (moins de 3 secondes) et indépendants de l'état de la base.

9.2 Tests unitaires

Service	Cas testés	Résultat
PromptsService (6 tests)	Prompt inexistant (404) ; contenu masqué pour un visiteur et pour un utilisateur non acheteur ; contenu visible pour le vendeur, pour un acheteur confirmé et pour un prompt gratuit, même sans connexion	6 / 6 réussis
PurchasesService (9 tests)	Refus d'acheter son propre prompt, un prompt gratuit ou déjà acheté ; création d'un achat PENDING et d'une session ; réutilisation d'un achat PENDING ; signature de webhook invalide ; achat confirmé et vendeur crédité ; idempotence d'un événement rejoué ; événement non géré ignoré	9 / 9 réussis
AdminUsersService (3 tests)	Utilisateur inexistant ; rôle invalide refusé ; rôle valide normalisé et enregistré	3 / 3 réussis

Résultat de l'exécution du 15/09/2026

```
$ npx jest --verbose
Test Suites: 3 passed, 3 total
Tests:      18 passed, 18 total
Time:       2.475 s
```

Exemple de test unitaire vérifiant l'idempotence du webhook :

apps/api/src/modules/purchases/purchases.service.spec.ts (extrait)

```
it('is idempotent: a second completed event for the same session is a no-op', async () => {
  const session = { id: 'cs_test_123' } as Stripe.Checkout.Session;
  stripeService.constructWebhookEvent.mockReturnValue({
    type: 'checkout.session.completed',
    data: { object: session },
  });
  findOnePurchase.mockResolvedValue({
    id: 'purchase-1', promptId: 'prompt-1', sellerId: 'seller-1',
    amount: '10.00', status: PurchaseStatus.COMPLETED, // achat déjà confirmé
  });

  await service.handleWebhookEvent(Buffer.from('{}'), 'sig');

  expect(savePurchase).not.toHaveBeenCalled();
  expect(usersService.save).not.toHaveBeenCalled(); // le vendeur n'est pas crédité
  // deux fois
  expect(promptsService.incrementSalesCount).not.toHaveBeenCalled();
});
```

Couverture de code. Les tests se concentrent sur les règles les plus critiques : 81 % des lignes de **PurchasesService**, 89 % de **AdminUsersService** et 44 % de **PromptsService** sont couvertes. La couverture globale de l'API reste faible (25 % des lignes), car les contrôleurs, l'authentification et les modules de catalogue ne sont pas encore testés unitairement. Étendre les tests à l'authentification est une amélioration prioritaire.

9.3 Jeu d'essai de la fonctionnalité la plus représentative : l'achat d'un prompt

J'ai choisi l'**achat d'un prompt** car cette fonctionnalité réunit les enjeux majeurs du projet : authentification, règles métier, protection du contenu payant, validation des données et intégration d'un service externe de paiement.

Conditions du test. Le 15/09/2026, sur l'API lancée en local avec Docker Compose. Un script crée deux comptes de test (un vendeur et un acheteur), valide leur email via Mailpit, puis publie deux prompts avec le compte vendeur : un prompt payant à 4,99 € et un prompt gratuit. Chaque scénario envoie une vraie requête HTTP et compare la réponse obtenue au résultat attendu.

Réf.	Scénario	Données en entrée	Résultat attendu	Résultat obtenu	Statut
T01	Un visiteur consulte la fiche d'un prompt payant	GET /prompts/:slug (sans cookie)	200, promptContent = null	200, promptContent = null	Conforme
T02	Un visiteur non connecté tente d'acheter	POST /purchases/checkout-session (sans cookie)	401 Unauthorized	401, code NO_TOKEN_PROVIDED	Conforme
T03	Le vendeur tente d'acheter son propre prompt	POST checkout-session, promptId = son prompt	400, CANNOT_PURCHASE_OWN_PROMPT	400, code CANNOT_PURCHASE_OWN_PROMPT	Conforme
T04	Un acheteur demande une session Stripe pour un prompt gratuit	POST checkout-session, prix = 0 €	400, PROMPT_IS_FREE (aucun appel à Stripe)	400, code PROMPT_IS_FREE	Conforme
T05	Un acheteur consulte un prompt gratuit	GET /prompts/:slug (prix = 0 €)	200, promptContent visible sans achat	200, promptContent visible	Conforme

Réf.	Scénario	Données en entrée	Résultat attendu	Résultat obtenu	Statut
T06	Données invalides envoyées à l'API	POST checkout-session, promptId = "pas-un-uuid"	400, rejet par le ValidationPipe	400, message « promptId must be a UUID »	Conforme
T07	Tentative d'injection d'un champ non prévu (prix)	POST checkout-session avec un champ amount = 0	400, propriété non autorisée (forbidNonWhitelisted)	400, message « property amount should not exist »	Conforme
T08	Un acheteur connecté achète un prompt payant	POST checkout-session, prompt à 4,99 €	201, URL de paiement Stripe Checkout	500, code CHECKOUT_SESSION_CREATION_FAILED	Écart
T09	État de l'achat avant confirmation	GET /purchases/by-session/:sessionId	200, statut PENDING, montant 4.99	Non exécuté (dépend de T08)	Bloqué
T10	L'acheteur clique une 2e fois sur « Acheter »	2e POST checkout-session	Même achat PENDING réutilisé	Non exécuté (dépend de T08)	Bloqué
T11	L'acheteur consulte le prompt avant confirmation	GET /prompts/:slug (achat PENDING)	200, promptContent = null	Non exécuté (dépend de T08)	Bloqué
T12	Un attaquant simule une confirmation de paiement	POST /purchases/webhook, signature falsifiée	400, INVALID_WEBHOOK_SIGNATURE, aucun achat validé	400, code INVALID_WEBHOOK_SIGNATURE	Conforme
T13	Le vendeur consulte son propre prompt payant	GET /prompts/:slug (cookie du vendeur)	200, promptContent visible	200, promptContent visible	Conforme

Bilan : 10 scénarios conformes, 1 écart (T08) et 3 scénarios bloqués par cet écart.

9.4 Analyse des écarts

Écart T08 : échec de la création de la session Stripe

Constat. La création d'une session de paiement renvoie une erreur 500 avec le code `CHECKOUT_SESSION_CREATION_FAILED` au lieu de l'URL de paiement attendue.

Analyse. Les logs de l'API montrent une erreur `StripeAuthenticationError` avec un statut HTTP 401 renvoyé par Stripe : la clé secrète Stripe de test configurée dans l'environnement local n'est plus valide. Il ne s'agit pas d'un défaut du code, mais d'un problème de configuration de l'environnement de développement.

Points positifs observés. L'erreur est interceptée : l'API renvoie un code d'erreur explicite, sans exposer de détail technique ni la clé au client, et les tests T01 à T07 prouvent que toutes les règles métier sont appliquées **avant** tout appel à Stripe.

Actions correctives. (1) Régénérer une clé secrète de test dans le tableau de bord Stripe et la renseigner dans le fichier d'environnement local ; (2) relancer T08 à T11 ; (3) rejouer un paiement complet avec une carte de test et la CLI Stripe (`stripe listen`) pour valider le webhook de bout en bout. Les scénarios de confirmation du paiement (webhook valide, crédit du vendeur, idempotence) sont en attendant couverts par les tests unitaires de la section 9.2, tous réussis.

Défaut mis en évidence par l'analyse de T08

En relisant le code pour analyser cet écart, j'ai constaté que l'achat **PENDING** est enregistré **avant** l'appel à Stripe. Si Stripe échoue, cette ligne reste en base avec un identifiant de session provisoire. Ce n'est pas bloquant — la même ligne est réutilisée à la tentative suivante — mais cela laisse une donnée incohérente si l'acheteur abandonne. Correction prévue : créer l'achat seulement après la réponse de Stripe, ou le passer à **FAILED** dans le bloc de gestion d'erreur.

10. Veille sur les vulnérabilités de sécurité

10.1 Démarche et sources

Tout au long du projet, j'ai mené une veille sur les vulnérabilités qui concernent les technologies utilisées. Cette veille m'a servi à la fois à choisir les mesures de protection et à vérifier régulièrement l'état des dépendances.

Source	Utilisation
OWASP Top 10 (2021)	Référentiel des dix risques les plus critiques des applications web, utilisé comme liste de contrôle
OWASP Cheat Sheet Series	Bonnes pratiques concrètes : stockage des mots de passe, JWT, envoi de fichiers, sessions
CERT-FR (ANSSI)	Avis et alertes de sécurité ; recommandations de l'ANSSI sur les mots de passe et le développement sécurisé
GitHub Advisory Database / npm audit	Vulnérabilités connues (CVE, GHSA) dans les paquets npm du projet
Documentation officielle	Sections sécurité de NestJS (Helmet, CORS, rate limiting), Stripe (webhooks) et Docker

10.2 OWASP Top 10 et mesures appliquées dans PromptVerse

Risque OWASP	Menace pour PromptVerse	Mesures en place	Reste à faire
A01 Contrôle d'accès défaillant	Lire un prompt payant sans l'acheter ; accéder à l'administration	Guard global, rôles, contenu masqué côté serveur, CORS restreint	Le rôle est lu dans le JWT : une rétrogradation ne prend effet qu'à l'expiration du jeton (24 h)
A02 Défaillances cryptographiques	Vol de mots de passe ou de sessions	bcrypt, HTTPS + HSTS, cookie <i>secure</i> , secrets hors du code	—
A03 Injection	Injection SQL, champs non prévus	Requêtes paramétrées TypeORM, DTO avec whitelist	—
A04 Conception non sécurisée	Fausse confirmation de paiement	Seul le webhook signé valide un achat ; idempotence	—
A05 Mauvaise configuration	En-têtes absents, Swagger exposé	Helmet, en-têtes Nginx, Swagger activable par variable d'environnement	—
A06 Composants vulnérables	Faible dans une dépendance	Audit des dépendances (10.3)	Mettre à jour les paquets signalés, automatiser l'audit dans la CI
A07 Authentification défaillante	Force brute, comptes non vérifiés	Throttling, email vérifié obligatoire, jetons à usage limité (15 min)	Mot de passe de 6 caractères minimum seulement : trop faible
A08 Intégrité des données	Modification d'un événement de paiement	Signature Stripe vérifiée sur le corps brut	—
A09 Journalisation	Attaque non détectée	Logs NestJS des erreurs Stripe et des signatures invalides	Centraliser les logs et alerter
A10 SSRF	Requêtes serveur vers des URL fournies	Aucune URL externe fournie par l'utilisateur n'est appelée	—

10.3 Audit des dépendances

Le 15/09/2026, j'ai lancé `npm audit --omit=dev` sur les deux applications, c'est-à-dire uniquement sur les dépendances embarquées en production.

Application	Critique	Élevée	Modérée	Faible	Total
Client React	0	0	0	0	0
API NestJS	0	14	1	1	16

Analyse des principales alertes de l'API :

Paquet	Vulnérabilité signalée	Impact pour PromptVerse	Action
multer (via @nestjs/platform-express)	Déni de service par des noms de champs multipart forgés	Réel : l'envoi d'images utilise multipart	Mettre à jour NestJS (version majeure) ; la limite de taille de Nginx réduit déjà l'exposition
nodemailer	Injection CRLF dans certains en-têtes d'email	Faible : les en-têtes concernés ne contiennent pas de données utilisateur	Mise à jour disponible sans changement majeur
qs	Contournement de la limite de tableaux, déni de service	Moyen : analyse des paramètres d'URL	<code>npm audit fix</code>
js-yaml , svgo , linkify-it , nanoid	Déni de service ou nettoyage SVG incomplet	Faible : dépendances indirectes non utilisées sur des données utilisateur	<code>npm audit fix</code>
joi	Pollution de prototype via des messages personnalisés	Très faible : Joi ne valide que la configuration interne	Mise à jour mineure

Plan d'action : appliquer les correctifs sans changement majeur (`npm audit fix`), planifier la montée de version de NestJS dans une branche dédiée avec exécution de tous les tests, puis ajouter une étape `npm audit --audit-level=high` au pipeline pour détecter automatiquement les nouvelles vulnérabilités.

10.4 Vulnérabilité identifiée dans mon propre code

En relisant le module d'envoi de fichiers à la lumière de l'**OWASP File Upload Cheat Sheet**, j'ai identifié une faiblesse. Le filtre accepte tout fichier dont le type MIME commence par `image`. Or ce type est **déclaré par le client** et peut être falsifié ; il accepte aussi `image/svg+xml`, alors qu'un fichier SVG peut contenir du JavaScript. Un fichier SVG malveillant, affiché directement depuis le domaine de l'application, pourrait exécuter un script (XSS stocké).

Mesures existantes qui limitent le risque : le nom du fichier est régénéré, la taille est limitée, et l'en-tête `X-Content-Type-Options: nosniff` empêche le navigateur de deviner un autre type.

Correction prévue :

Correctif proposé

```
const ALLOWED_TYPES = ['image/jpeg', 'image/png', 'image/webp']; // liste blanche stricte, sans SVG

fileFilter: (req, file, cb) => ALLOWED_TYPES.includes(file.mimetype)
  ? cb(null, true)
  : cb(new BadRequestException({ code: ErrorCodes.UNSUPPORTED_FILE_TYPE }), false),

// + après l'écriture : vérifier la signature binaire réelle du fichier (« magic bytes »)
//   et forcer l'extension en fonction du type détecté, pas du nom d'origine.
```

11. Situation de travail ayant nécessité une recherche anglophone

11.1 Le problème rencontré

Lors de l'intégration du webhook Stripe, chaque événement envoyé par Stripe était rejeté par mon API, alors que le secret du webhook était correct. La bibliothèque Stripe renvoyait l'erreur suivante :

```
Webhook signature verification failed. Err: No signatures found matching the expected signature for payload.
```

Sans vérification de signature, n'importe qui pourrait envoyer une fausse confirmation de paiement : il n'était donc pas question de désactiver ce contrôle.

11.2 La démarche de recherche

- J'ai d'abord cherché le message d'erreur exact sur Google, en anglais, ce qui m'a mené à la documentation de Stripe (docs.stripe.com/webhooks/signature) et à des discussions GitHub sur la bibliothèque stripe-node.
- La documentation de Stripe explique que la signature est calculée sur le corps **exact** de la requête : si un framework analyse le JSON puis le re-séréalise, un seul espace ou un ordre de clés différent suffit à faire échouer la vérification.
- J'ai ensuite cherché comment obtenir ce corps brut avec NestJS (mots-clés : *nestjs raw body stripe webhook*), ce qui m'a conduit à la page « Raw body » de la documentation officielle de NestJS.

11.3 Extrait de la source et traduction

Source : documentation officielle NestJS, page « Raw body » (docs.nestjs.com/faq/raw-body).

"One of the most common use-case for having access to the raw request body is performing webhook signature verifications."

Traduction : « L'un des cas d'usage les plus courants de l'accès au corps brut de la requête est la vérification de la signature des webhooks. »

La documentation précise ensuite que ce calcul de signature (un hachage HMAC) nécessite le corps de la requête **non désérialisé**, et que NestJS peut le conserver si l'on active l'option **rawBody** à la création de l'application ; le corps brut est alors disponible sous forme de **Buffer** dans **req.rawBody**, en plus du corps JSON habituel.

11.4 La solution appliquée

NestJS analysait automatiquement le JSON de toutes les requêtes : la bibliothèque Stripe recevait donc un objet re-sérialisé, et non le texte exact signé par Stripe. J'ai activé l'option `rawBody` et transmis `req.rawBody` à la vérification :

```
// main.ts : rawBody conserve le Buffer brut de chaque requête sur req.rawBody,
// en plus du parsing JSON habituel (utilisé par toutes les autres routes)
const app = await NestFactory.create<NestExpressApplication>(AppModule, {
  rawBody: true,
});

// purchases.controller.ts
async handleWebhook(@Req() req: RawBodyRequest<Request>,
  @Headers('stripe-signature') signature: string | undefined) {
  await this.purchasesService.handleWebhookEvent(req.rawBody, signature);
}

// stripe.service.ts
constructWebhookEvent(payload: Buffer, signature: string): Stripe.Event {
  return this.client.webhooks.constructEvent(
    payload, signature, this.configService.getOrThrow<string>('STRIPE_WEBHOOK_SECRET'),
  );
}
```

Après cette correction, les événements signés par Stripe ont été acceptés et les requêtes falsifiées sont restées rejetées (scénario T12 du jeu d'essai). Cette recherche m'a appris à lire attentivement la documentation officielle en anglais et à comprendre la cause d'une erreur plutôt que de contourner un contrôle de sécurité.

12. Déploiement et démarche DevOps

12.1 Conteneurisation

Chaque application possède un **Dockerfile multi-stage** : une première étape installe les dépendances et compile le code, la dernière ne contient que ce qui est nécessaire à l'exécution. Les images sont plus légères et n'embarquent ni le code source TypeScript ni les dépendances de développement.

apps/api/Dockerfile (extrait)

```
FROM node:22-alpine AS build          # 1. compilation TypeScript
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM node:22-alpine AS deps           # 2. dépendances de production uniquement
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci --omit=dev && npm cache clean --force

FROM node:22-alpine AS production     # 3. image finale
ENV NODE_ENV=production
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY --from=build /app/dist ./dist
COPY docker-entrypoint.sh /usr/local/bin/docker-entrypoint.sh
USER node                             # le processus ne tourne pas en root
HEALTHCHECK CMD wget -q -O - "http://127.0.0.1:3000/${API_PREFIX:-api}/health" || exit 1
ENTRYPOINT ["docker-entrypoint.sh"]
CMD ["node", "dist/main"]
```

Le client React est compilé avec Vite dans une première étape, puis servi par une image Nginx légère. Au démarrage du conteneur de l'API, un script attend que MySQL soit joignable, exécute les migrations, puis lance l'application :

apps/api/docker-entrypoint.sh (extrait simplifié)

```
echo "[entrypoint] attente de ${DB_HOST}:${DB_PORT} ..."
while ! node -e "require('net').connect({host:'${DB_HOST}',port:${DB_PORT}})
  .on('connect',()=>process.exit(0)).on('error',()=>process.exit(1));" do
  i=$((i + 1)); [ "$i" -ge 60 ] && echo "base injoignable" >&2 && exit 1
  sleep 2
done
echo "[entrypoint] exécution des migrations ..."
node ./node_modules/typeorm/cli.js -d dist/database/data-source.js migration:run
exec "$@"
```

12.2 Intégration et déploiement continu

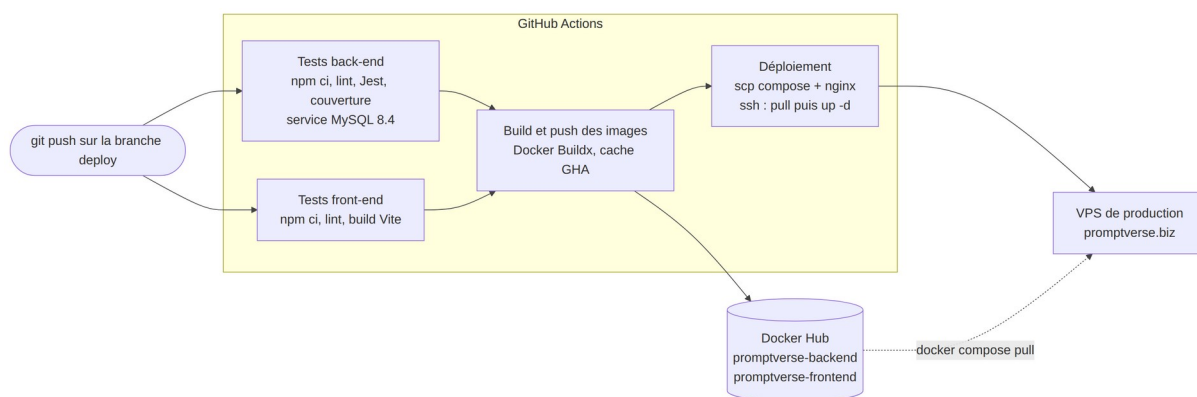


Figure 22 — Pipeline CI/CD GitHub Actions

Le pipeline est déclenché par un push ou une pull request sur la branche **deploy** :

1. **Tests back-end** : installation, lint, tests Jest et couverture, avec un service MySQL 8.4 disponible dans le job.
2. **Tests front-end** : installation, lint et build de production Vite.
3. **Build et push** (si les deux jobs précédents réussissent) : construction des images de l'API et du client avec Docker Buildx et le cache de GitHub Actions, puis publication sur Docker Hub.
4. **Déploiement** : copie du `docker-compose.prod.yml` et de la configuration Nginx sur le VPS, puis connexion SSH pour récupérer les nouvelles images et redémarrer les conteneurs.

Les secrets (identifiants Docker Hub, accès SSH au serveur) sont stockés dans les **secrets GitHub** et ne figurent jamais dans le dépôt. Les secrets applicatifs (JWT, Stripe, base de données) restent dans le fichier d'environnement du serveur.

12.3 Architecture de production

Service	Rôle en production	Persistance
nginx	Point d'entrée unique : HTTPS, redirection HTTP → HTTPS, en-têtes de sécurité, limitation de débit, reverse proxy	Certificats Let's Encrypt montés en lecture seule
api	Image Docker Hub de l'API NestJS ; migrations exécutées au démarrage	Volume <code>uploads_data</code> pour les images envoyées
client	Image Docker Hub du front-end (fichiers statiques servis par Nginx)	—
database	MySQL 8.4, non exposé sur Internet	Volume <code>mysql_data</code>
redis	Redis 7, non exposé sur Internet	Volume <code>redis_data</code>

Seul Nginx publie des ports (80 et 443). La base de données et Redis ne sont accessibles que sur le réseau interne Docker. Les certificats HTTPS sont obtenus avec **Certbot** (validation par le dossier `.well-known/acme-challenge` servi par Nginx).

12.4 Procédure de déploiement

Premier déploiement

1. Préparer le VPS : installer Docker et Docker Compose, ouvrir uniquement les ports 22, 80 et 443.
2. Pointer le nom de domaine `promptverse.biz` vers l'adresse IP du serveur.
3. Créer sur le serveur le fichier d'environnement de production (secrets JWT et Stripe, accès à la base, URL publiques).
4. Obtenir le certificat HTTPS avec Certbot.
5. Renseigner les secrets GitHub (Docker Hub, hôte et clé SSH), puis pousser sur la branche `deploy`.
6. Déclarer l'URL du webhook (`https://promptverse.biz/api/v1/purchases/webhook`) dans le tableau de bord Stripe.

Mises à jour

Une mise à jour consiste à fusionner les changements dans la branche `deploy`. Le pipeline teste, construit, publie et déploie automatiquement ; les migrations sont appliquées au redémarrage de l'API. En cas de problème, il suffit de redéployer l'image précédente, car chaque image est également publiée avec une étiquette correspondant au commit.

13. Bilan et perspectives

13.1 Bilan du projet

Toutes les fonctionnalités du périmètre prévu ont été livrées et l'application est en production : inscription avec vérification d'email, catalogue, publication, achat avec Stripe, prompts gratuits, tableau de bord et administration. Les choix de conception les plus structurants — cookie httpOnly, protection par défaut des routes, contenu payant masqué côté serveur, webhook idempotent — ont été pensés dès le départ et vérifiés par des tests.

Le projet a aussi connu des ajustements. Le stockage des fichiers a d'abord reposé sur MinIO, puis sur SeaweedFS, avant que je choisisse un simple stockage sur disque avec un volume Docker : les solutions compatibles S3 étaient surdimensionnées pour quelques images et compliquaient le déploiement. Cette décision m'a appris à privilégier la solution la plus simple qui répond réellement au besoin.

13.2 Améliorations prévues

Priorité	Amélioration	Justification
Haute	Mettre en cache dans Redis les lectures publiques (catégories, outils IA, catalogue) avec invalidation	Exploiter réellement l'infrastructure NoSQL en place et réduire la charge de MySQL
Haute	Liste blanche stricte des types d'images et contrôle de la signature binaire	Corriger la faiblesse identifiée en 10.4
Haute	Mettre à jour les dépendances et ajouter <code>npm audit</code> au pipeline	Traiter les 16 alertes de l'audit (10.3)
Haute	Renforcer la politique de mot de passe (12 caractères minimum)	Recommandations de l'ANSSI et de l'OWASP
Moyenne	Étendre les tests (authentification, contrôleurs) et ajouter des tests End-to-End	Couverture globale actuelle de 25 %
Moyenne	Enregistrer l'achat après la réponse de Stripe ou le passer à FAILED en cas d'erreur	Défaut mis en évidence par le jeu d'essai (9.4)
Moyenne	Anonymiser les données personnelles à la suppression du compte	Respect du droit à l'effacement (RGPD)
Évolution	Avis et notes, liste de favoris, recherche plein texte	Fonctionnalités déjà spécifiées dans OpenSpec
Évolution	Reversement des gains aux vendeurs avec Stripe Connect	Permettre aux vendeurs de retirer leur solde

14. Conclusion

PromptVerse a été pour moi un projet complet et exigeant, mené seul de bout en bout. Partir d'un besoin, le formaliser en user stories et en spécifications, le modéliser avec UML et Merise, puis le transformer en une application testée et déployée m'a fait parcourir tout le cycle de vie d'une application, et pas seulement sa partie la plus visible.

Sur le plan technique, j'ai appris à structurer une API en couches avec NestJS et à considérer la sécurité comme une composante de la conception : protection par défaut des routes, cookie httpOnly, masquage du contenu payant, idempotence du webhook. Côté client, j'ai progressé sur l'organisation d'une application React réelle, en séparant les données serveur (TanStack Query) de l'état de l'interface (Zustand).

Travailler avec des spécifications écrites avant le code m'a montré l'intérêt d'une démarche outillée : les critères d'acceptation deviennent les tests, et le développement cesse d'être une interprétation du besoin. Enfin, prendre en charge le déploiement et l'intégration continue m'a fait comprendre le point de vue DevOps : un projet n'est terminé que lorsqu'il fonctionne en production, de façon reproductible.

Le jeu d'essai et la veille de sécurité m'ont aussi montré les limites actuelles du projet, que j'assume et que j'ai planifiées : usage réel du cache Redis, durcissement de l'envoi de fichiers, mise à jour des dépendances et extension des tests. Le socle est solide et a été conçu pour accueillir ces évolutions sans être réécrit.

Annexes

Annexe A — Principales routes de l'API

Méthode et route	Accès	Description
POST /api/v1/auth/register	Public	Inscription et envoi de l'email de vérification
GET /api/v1/auth/verify-email/:id/:token	Public	Vérification de l'adresse email
POST /api/v1/auth/login	Public	Connexion et dépôt du cookie de session
POST /api/v1/auth/logout	Connecté	Déconnexion (suppression du cookie)
POST /api/v1/auth/forgot-password	Public	Envoi d'un lien de réinitialisation
POST /api/v1/auth/reset-password	Public	Nouveau mot de passe
GET / PATCH / DELETE /api/v1/users/me	Connecté	Profil, modification, suppression du compte
POST / DELETE /api/v1/users/avatar	Connecté	Envoi ou suppression de l'avatar
GET /api/v1/categories et /api/v1/ai-tools	Public	Listes des catégories et des outils IA
GET /api/v1/prompts	Public	Catalogue paginé (sans contenu payant)
GET /api/v1/prompts/category/:slug et /ai-tool/:slug	Public	Prompts d'une catégorie ou d'un outil IA
GET /api/v1/prompts/:slug	Public (authentification facultative)	Fiche d'un prompt, contenu selon les droits
GET /api/v1/prompts/me	Connecté	Prompts publiés par l'utilisateur
POST /api/v1/prompts	Connecté	Publication d'un prompt (multipart, images facultatives)
POST /api/v1/purchases/checkout-session	Connecté	Création d'une session Stripe (refusée pour un prompt gratuit)
POST /api/v1/purchases/webhook	Stripe (signature)	Confirmation ou expiration d'un paiement
GET /api/v1/purchases/mine	Connecté	Historique des achats
GET /api/v1/purchases/by-session/:sessionId	Connecté	État d'un achat après retour de Stripe
GET /api/v1/uploads/:folder/:filename	Public	Lecture d'une image envoyée
GET /api/v1/admin/users	ADMIN, SUPER_ADMIN	Liste des utilisateurs
PATCH /api/v1/admin/users/:userId/make-role	SUPER_ADMIN	Modification du rôle d'un utilisateur
POST / PATCH / DELETE /api/v1/admin/categories	ADMIN, SUPER_ADMIN	Gestion des catégories (archivage et restauration)
POST / PATCH / DELETE /api/v1/admin/ai-tools	ADMIN, SUPER_ADMIN	Gestion des outils IA (archivage et restauration)
GET /api/health	Public	Contrôle de santé utilisé par Docker
GET /api/docs	Développement	Documentation Swagger / OpenAPI

Annexe B — Organisation du dépôt

promptVerse/	
├── apps/	
│ ├── api/	API NestJS
│ │ ├── src/common/	guards, décorateurs, codes d'erreur, validation de la
│ ├── configuration	
│ │ ├── src/modules/	auth, users, prompts, purchases, categories, ai-tools, admin,
│ ├── storage, mail	
│ │ ├── src/database/	data-source et migrations TypeORM
│ │ ├── Dockerfile	
│ │ └── docker-entrypoint.sh	
│ └── client/	application React
│ ├── src/pages/	écrans
│ ├── src/components/	composants réutilisables
│ ├── src/hooks/	requêtes TanStack Query
│ ├── src/common/	client API, stores Zustand, types
│ └── Dockerfile	
├── docs/	cahier des charges, UML, Merise, maquettes, sprints, backlog
├── openspec/changes/	spécifications par module
├── nginx/nginx.prod.conf	configuration Nginx de production
├── docker-compose.yml	environnement de développement
├── docker-compose.prod.yml	environnement de production
└── .github/workflows/deploy.yml	pipeline CI/CD

Annexe C — Glossaire

Terme	Définition
API REST	Interface qui expose des ressources par des URL et des méthodes HTTP (GET, POST, PATCH, DELETE).
DTO	Data Transfer Object : objet qui décrit et valide les données reçues par l'API.
Guard	Classe NestJS qui autorise ou refuse l'accès à une route avant son exécution.
JWT	JSON Web Token : jeton signé qui contient l'identité de l'utilisateur.
Cookie httpOnly	Cookie envoyé automatiquement par le navigateur mais illisible par le JavaScript de la page.
Webhook	Requête HTTP envoyée par un service externe (ici Stripe) pour signaler un événement.
Idempotence	Propriété d'une opération qui produit le même résultat qu'elle soit exécutée une ou plusieurs fois.
ORM	Outil qui fait correspondre des classes du code à des tables de la base de données (ici TypeORM).
Migration	Script versionné qui fait évoluer le schéma de la base de données.
NoSQL clé-valeur	Base de données qui associe des valeurs à des clés, sans schéma relationnel (ici Redis).
CI/CD	Intégration et déploiement continu : tests, construction et mise en production automatisés.
Multi-stage build	Dockerfile en plusieurs étapes qui ne garde dans l'image finale que le nécessaire.